

VERIQTA



AWS

SYSTEM DESIGN & PRODUCTION

INTERVIEW HANDBOOK



ADVANCED EDITION

Master real-world AWS architecture, production challenges, and system design interviews at a senior engineer level.



SYSTEM DESIGN AT SCALE

Design Netflix, Uber, Banking Systems & more.



GLOBAL ARCHITECTURE & RESILIENCE

Multi-Region, Multi-AZ, DR, and High Availability.



COST & PERFORMANCE OPTIMIZATION

Make smart trade-offs and optimize for scale.



SECURITY & COMPLIANCE

Deep dives into security incidents, zero trust, IAM, and governance.



PRODUCTION CHALLENGES

Outages, bottlenecks, scaling issues & more.



REAL INTERVIEW PREP

Senior-level questions with model answers and explanations.



REAL SCENARIOS

Real postmortems & case studies



SENIOR-LEVEL INTERVIEWS

Advanced questions & model answers



ARCHITECTURE TRADE-OFFS

Deep dives into decision making



MIGRATION STRATEGIES

Lift & Shift, Replatform, Refactor & more



BEST PRACTICES AT SCALE

Operational excellence, governance & more



— THINK LIKE AN ARCHITECT. SOLVE LIKE AN ENGINEER. DELIVER LIKE A LEADER. —



VERIQTA

ORIGINAL CONTENT • COPYRIGHT PROTECTED



AWS INTERVIEW ROADMAP

Your guide to cracking real-world AWS system design & production interviews

**GOAL:**

Think like a Senior Engineer.
Design like an Architect.
Solve like an Owner.

**AWS INTERVIEW STAGES****1. SCREENING**

Resume screening & recruiter call

**2. TECHNICAL FOUNDATION**

Core AWS services, concepts & basics

**3. SYSTEM DESIGN ROUND**

Design scalable, resilient systems

**4. DEEP DIVE**

Troubleshooting, optimization & trade-offs

**5. BEHAVIORAL ROUND**

Leadership, ownership & real-world experience

WHAT COMPANIES ACTUALLY TEST**Scalability & High Availability**

Can you design systems that scale and never go down?

**Reliability & Fault Tolerance**

Can you handle failures, spikes, and regional outages?

**Cost Optimization**

Can you make smart architectural trade-offs?

**Security & Compliance**

Do you build secure, compliant, and auditable systems?

**Performance & Troubleshooting**

Can you identify bottlenecks and resolve issues?

**Architecture Trade-offs**

Do you understand the "why" behind decisions?

FAANG vs STARTUPS vs CONSULTING**FAANG**

- Focus on scale, reliability, and complex architectures
- Deep dive on AWS services & edge cases
- Expect detailed trade-off discussions
- Strong emphasis on ownership & impact

STARTUPS

- Focus on speed, cost, and getting to MVP
- Expect you to wear multiple hats
- Practical, hands-on problem solving
- Lean architectures & rapid iteration mindset

CONSULTING

- Focus on best practices, frameworks & design patterns
- Case studies & client scenarios
- Clear communication and structured thinking
- Emphasis on business impact & ROI

WHAT THIS HANDBOOK COVERS**Real Interview Questions**

Curated from top companies

**Production Scenarios**

Based on real outages & incidents

**System Design Examples**

Netflix, Uber, Banking & more

**Architecture Trade-offs**

Understand the why, not just the how

**Postmortem-Style Cases**

Learn from real-world failures

**Model Answers & Diagrams**

Explain like a Senior Engineer

AWS ARCHITECTURE OVERVIEW (AT A GLANCE)**FRONTEND**

Route 53
CloudFront

EDGE & NETWORK

VPC, Subnets
IGW, NAT Gateway
Route Tables
Security Groups
NACLs

COMPUTE

EC2, ECS, EKS
Lambda
Auto Scaling
Elastic Beanstalk

DATA LAYER

RDS, Aurora
DynamoDB
ElastiCache
S3, EFS, FSx

INTEGRATION

API Gateway
SQS, SNS
EventBridge
Step Functions

OBSERVABILITY & SECURITY

CloudWatch, CloudTrail
AWS Config, X-Ray
IAM, KMS, Secrets Manager
GuardDuty, Security Hub

★ SUCCESS MINDSET

It's not about memorizing services. It's about solving real problems at scale with the right architecture and trade-offs.



Understand
Requirements



Design
Option(s)



Evaluate
Trade-offs



Recommend
& Justify

THE AWS INTERVIEW LANDSCAPE

WHAT SENIOR ENGINEERS ARE REALLY EVALUATED ON



THINK BIG
Design for scale and resilience



OWN IT
Take ownership like an engineer



DEEP DIVE
Go beyond basics, dive into trade-offs

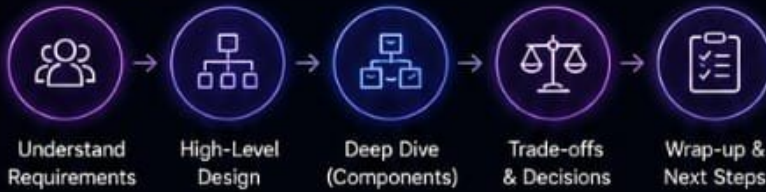


DELIVER VALUE
Solve real problems that matter



COMMUNICATE
Explain clearly, influence decisions

SYSTEM DESIGN INTERVIEW PRINCIPLES



WHAT INTERVIEWERS LOOK FOR

- Structured thinking & clarity
- Scalable, reliable & cost-effective designs
- Deep AWS knowledge & best practices
- Trade-off analysis & decision making
- Operational excellence & monitoring
- Security, compliance & resilience mindset
- Real-world experience & ownership

FAANG vs STARTUPS vs CONSULTING: WHAT'S DIFFERENT?

	FAANG	STARTUPS	CONSULTING
Focus	Massive scale, global systems	Speed, agility, ship fast	Frameworks, patterns, best practices
Scale	Millions to billions of users	Thousands to millions of users	Varies by client (1K to 100M+)
Depth	Deep dive into non-functional requirements	Balanced depth with rapid iteration	Broad knowledge across domains
Trade-offs	Complex trade-offs, cost vs performance	Speed vs cost simplicity first	Business impact, cost & risk
Ownership	High - you own the outcome	Medium - move fast, iterate	Medium - client impact first
Expectations	Flawless design, detail oriented	Good enough that can scale	Framework driven, structured
Communication	Clear, crisp, data-driven	Practical, simple, impact focused	Executive friendly, storytelling

KEY AREAS YOU MUST MASTER (ADVANCED LEVEL)

System Design at Scale Design highly available, elastic & fault tolerant systems	Security & Compliance Zero trust, encryption, IAM, logging & compliance
Multi-Account & Organizational Strategy Secure, scalable & governable AWS Organizations	Production Operations Monitoring, alerting, logging, incident response
Disaster Recovery Cross-region, RPO/RTO, failover strategies	Migration Strategies Lift & shift, replatform, refactor, re-architect
Performance Engineering Latency, throughput, caching & bottlenecks	Architecture Trade-offs Make the right calls with clear reasoning
Cost Optimization Architect for cost efficiency without compromising scale	Postmortem Thinking Learn from failures, improve systems

REAL-WORLD SCENARIOS YOU WILL BE TESTED ON



“Senior engineers don't just know services. They design systems that scale, survive, and create real business impact.”

THE MINDSET OF A SENIOR AWS ENGINEER



Business First
Understand the problem deeply



Think in Systems
End-to-end thinking



Question Everything
Challenge assumptions & constraints



Design for Change
Build for failure, expect change



Measure & Improve
Observe, learn, iterate



Own the Outcome
Take responsibility, deliver value



DESIGNING NETFLIX ON AWS

High availability. Massive scale.
Global reach.

INTERVIEWER WILL TEST:

- ✓ Requirements gathering & assumptions
- ✓ Scalability, availability & fault tolerance
- ✓ Global architecture & edge delivery
- ✓ Data partitioning & replication strategy
- ✓ Caching, performance & cost trade-offs
- ✓ Security, monitoring & operational excellence

ASSUMPTIONS

- 500M+ active users globally
- Millions of concurrent streams
- High read, low write workload
- 99.99% availability
- Low latency worldwide (<2s start time)
- Global content library with personalization



HIGH LEVEL ARCHITECTURE



DESIGN HIGHLIGHTS

- ✓ CloudFront delivers content from edge locations for ultra-low latency.
- ✓ S3 stores original & encoded videos with lifecycle policies.
- ✓ Microservices architecture for independent scaling.
- ✓ DynamoDB global tables for multi-region active-active data.
- ✓ Aurora Multi-AZ for high availability.
- ✓ Personalization with ML powered by Amazon Personalize.
- ✓ Auto Scaling ensures seamless handling of traffic spikes.
- ✓ WAF + Shield for security and DDoS protection.

KEY TRADE-OFFS & DECISIONS

DECISION	OPTIONS	CHOSEN	WHY?
Content Delivery	S3 + CloudFront vs Direct S3	CloudFront	Lower latency, global reach, caching & cost-effective
Database	DynamoDB vs RDS	DynamoDB	Massive scale, low latency, serverless, global tables
Search	OpenSearch vs Elasticsearch self-managed	OpenSearch	Managed, scalable, integrates with AWS
Transcoding	EC2 vs Elemental MediaConvert	MediaConvert	Managed, reliable, highly optimized
Architecture	Monolith vs Microservices	Microservices	Independent scaling, fault isolation, agility

COMMON INTERVIEW QUESTIONS

- How would you ensure low latency for video streaming worldwide?
- How would you handle a sudden 10x traffic spike during a live event?
- How would you design for high availability and disaster recovery?
- How would you personalize recommendations for users?
- How would you reduce costs while maintaining performance?

KEY TAKEAWAYS

- ★ **Think global:** Use edge locations, multi-region & global services.
- ★ **Optimize for read-heavy:** Leverage caching, CDN and NoSQL databases.
- ★ **Design for failure:** Multi-AZ, auto scaling, health checks.
- ★ **Observe everything:** Monitoring, logging and tracing are non-negotiable.
- ★ **Balance cost & performance:** Use right service for the right job.

SCALE NUMBERS (EXAMPLE)

Videos in library	100,000+
Concurrent streams	Millions
Data transfer per day	Petabytes
Requests per second	Millions
Regions	25+
Edge locations	400+

PRO TIP

Always start with requirements, make reasonable assumptions, draw a high-level architecture, then dive deep into each component explaining the why behind your choices and trade-offs.

Think out loud. Communicate clearly. Justify your decisions.

DESIGNING UBER ON AWS

Real-time ride-hailing at massive scale.
Low latency. High availability. High concurrency.

ASSUMPTIONS

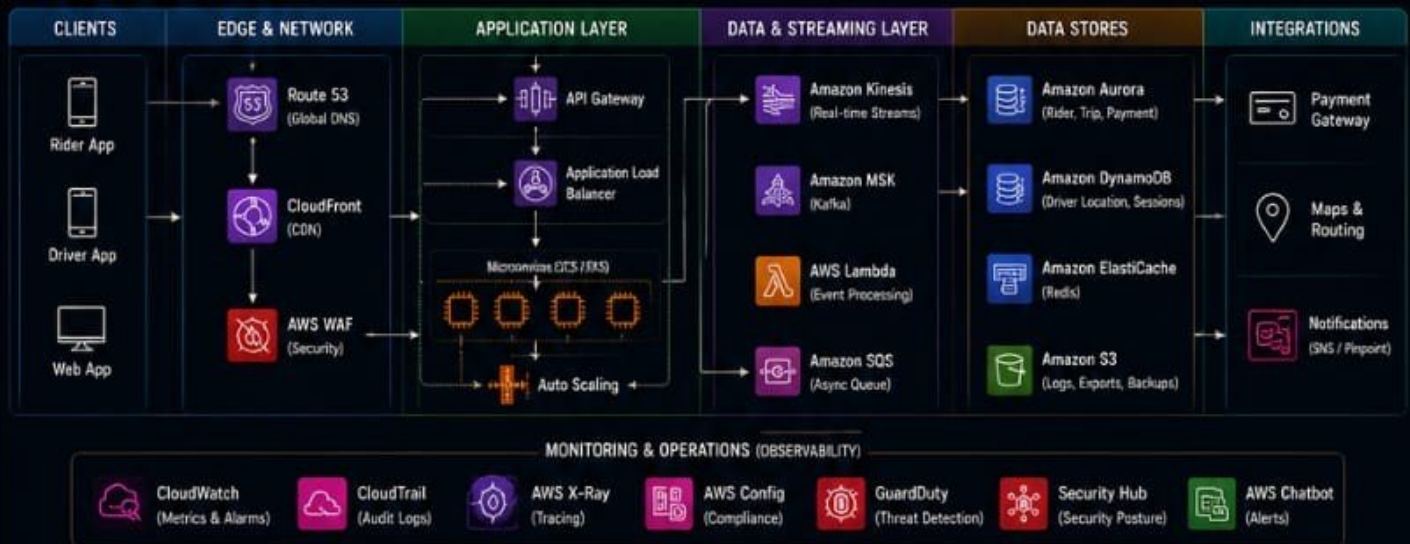
- 100M+ users and drivers globally
- Millions of ride requests per minute
- Real-time location tracking
- Sub-second matching & responses
- Highly available, globally distributed
- Supports surge pricing & dynamic scaling

INTERVIEWER WILL TEST:

- System design & decomposition
- Real-time data pipelines
- Scalability & performance
- High availability & fault tolerance
- Geo-distribution & latency
- Cost optimization trade-offs
- Operational excellence

PAGE
4

HIGH LEVEL ARCHITECTURE



CORE SERVICES & THEIR PURPOSE

Service	Why We Use It
Route 53 + CloudFront	Global DNS resolution and caching to reduce latency globally.
API Gateway	Secure, scalable API entry point for all clients.
ALB	Distributes traffic across microservice instances.
ECS / EKS	Container orchestration for microservices.
Kinesis + MSK	Real-time streaming of location and trip events.
DynamoDB	Low-latency, high-throughput for volatile data (locations, sessions).
Aurora (Multi-AZ)	Strongly consistent relational data with high availability.
ElastiCache (Redis)	Caching, session store, and fast lookups.
S3	Durable storage for logs, analytics, and backups.
Lambda	Event-driven processing and lightweight workflows.
SQS	Decoupling and buffering between services.

KEY DESIGN CONSIDERATIONS

- Low Latency**
Use CloudFront, Regional services, caching, and in-memory stores.
- Real-time Matching**
Use streaming (Kinesis/MSK) + in-memory indexes (ElastiCache) for fast driver-rider matching.
- High Availability**
Multi-AZ for data stores, Auto Scaling, and health checks for services.
- Geo-Distribution**
Deploy in multiple regions with active-active architecture for global availability.
- Cost Optimization**
Right-size resources, use Spot capacity for worker nodes, lifecycle policies on S3.
- Security**
WAF, Shield, encryption, least privilege IAM, Secrets Manager.

DATA FLOW (TRIP REQUEST JOURNEY)

- Rider opens app and requests a ride.
- Request hits Route 53 → CloudFront → API Gateway.
- Request is routed to microservices (ECS/EKS).
- Trip request event is published to Kinesis/MSK.
- Matching service consumes stream, queries live driver locations (DynamoDB/Redis).
- Best driver is selected and notified via SNS / SQS.
- Trip updates are streamed in real-time.
- Data is persisted in Aurora, analytics in S3.

Goal: Match the closest available driver in <1 second.

SCALE STRATEGIES

- Horizontal Scaling**
Auto Scaling groups for microservices (ECS/EKS).
- Database Scaling**
Aurora Read Replicas, DynamoDB auto scaling.
- Caching**
Use ElastiCache + CloudFront to offload backends.
- Event-Driven**
Decouple services with SQS, SNS, and Lambda.

TRADE-OFFS DISCUSSED IN INTERVIEWS

Decision	Option A	Option B	Trade-off
Database	Aurora	DynamoDB	Aurora for relational consistency, DynamoDB for massive scale & low latency.
Messaging	Kinesis	MSK (Kafka)	Kinesis is serverless, MSK offers more control & flexibility.
Compute	ECS (Fargate)	EKS (EC2 + Spot)	Fargate is simpler, Spot is cheaper but less predictable.
Caching	ElastiCache (Redis)	DAX + App Cache	Redis is versatile, DAX is DynamoDB-specific.
Global Strategy	Active-Active	Active-Passive	Active-Active is complex but best for low latency & resilience.

POTENTIAL IMPROVEMENTS

- ML-based ETA prediction using SageMaker.
- Fraud detection with real-time analytics (Kinesis + SageMaker + Lookout for Metrics).
- Data lake for analytics using S3 + Glue + Athena.
- Blue/Green or Canary deployments for safe releases.
- Cross-region replication for zero-data-loss.

COMMON INTERVIEW QUESTIONS

- How would you design Uber to handle 10x traffic during peak hours?
- How would you ensure real-time location updates at scale?
- How would you reduce the matching latency to sub-second?
- How would you design for high availability across regions?
- How would you handle a database outage?
- How would you reduce operational costs?

PRO TIP

Always clarify assumptions, break the problem down, use the AWS Well-Architected Framework, and explain the "why" behind every decision.

Think like an Architect. Communicate like a Leader.

WELL-ARCHITECTED ALIGNMENT



MULTI-ACCOUNT AWS ORGANIZATIONS

Design secure, scalable and governed AWS environments using AWS Organizations.

WHY MULTI-ACCOUNT?

- Strong isolation between workloads and teams
- Better security and blast radius reduction
- Centralized governance and compliance
- Simplified billing and cost allocation
- Enables scalable landing zone and DevSecOps
- Essential for enterprise and AWS Well-Architected

AWS ORGANIZATIONS HIERARCHY (EXAMPLE)



ACCOUNT STRATEGY



One workload per account
Maximum isolation and flexibility.



Environment separation
Prod, Staging, Dev in separate accounts.



Centralized security
Log archive, audit, and security tooling centralized.



Shared services
Network, DNS, IAM Identity Center, backup, monitoring.



Future-proof
Easily add new workloads, regions, or teams.

COMMON LANDING ZONE PATTERNS

1 Consolidated

All accounts in one OU. Simple but less scalable.



2 Business Unit

Separate OUs per BU. Good for medium to large organizations.



3 Env-Based

Separate OUs for Prod, Non-Prod, Sandbox.



4 Hybrid

Combine Business Unit and Env-Based for large enterprises.



GUARDRAILS & GOVERNANCE



Service Control Policies (SCPs)
Set permissions guardrails for accounts/OUs. Do not use for everything.



Tag Policies
Enforce consistent tagging across all accounts.



AWS Config
Evaluate resource compliance across accounts.



AWS CloudTrail
Centralize trails in log archive account.



AWS Security Hub
Aggregate and prioritize security findings.



AWS Control Tower
Automate landing zone setup and governance.

CROSS-ACCOUNT ACCESS OPTIONS



IAM Roles
AssumeRole for users, apps and services.



AWS IAM Identity Center (SSO)
Single sign-on across all accounts.



Resource Policies
S3 bucket, KMS key, SNS topic, etc.



External Identity Provider
SAML/OIDC federation for enterprise access.



BEST PRACTICE
Use IAM Identity Center + Permission Sets for human access. Use Roles for apps.

INTERVIEW SCENARIOS

- You need to onboard a new business unit with strict isolation. How would you design the account structure?
What to cover: New OU, separate accounts, SCPs, shared services, IAM Identity Center.
- How would you ensure that developers cannot delete critical resources in the production accounts?
What to cover: SCPs, IAM permissions boundaries, least privilege, break-glass access.
- Logs must be centralized and immutable across all accounts. How would you achieve this?
What to cover: CloudTrail org trail, log archive account, S3 Object Lock, KMS.
- How would you implement a secure CI/CD pipeline that deploys across multiple accounts?
What to cover: CodePipeline/CodeBuild roles, cross-account role assumption, artifact bucket.
- The security team needs read-only access to all accounts. How would you implement this?
What to cover: IAM Identity Center, permission sets, read-only roles.
- How would you prevent usage of specific AWS services in sandbox accounts?
What to cover: SCPs deny list, service restrictions, tag policies.

KEY TAKEAWAYS

- Multi-account is the foundation of security and scale.
- Design for isolation, governance, and operational excellence.
- Use SCPs wisely. They are guardrails, not permissions.
- Centralize logs, security, and identity.
- Automate account provisioning and governance.
- Follow AWS Well-Architected Framework and Landing Zone best practices.

REMEMBER

Well-Architected Framework is your compass. Organizations is your foundation. Governance is your safety net.

COMMON SCP EXAMPLES

USE CASE	SCP EXAMPLE (DESCRIPTION)	EFFECT
Deny root user usage	Deny actions if <code>aws:username = "root"</code>	Prevent root user sign-in and API usage
Disable IAM user creation	Deny <code>iam:CreateUser</code> , <code>iam:CreateAccessKey</code>	Enforce federated access via SSO
Restrict regions	Deny all except approved regions	Control where resources can be created
Deny critical services	Deny <code>ec2:TerminateInstances</code>	Protect critical resources
Enforce tagging	Deny if <code>aws:RequestTag/Environment</code> is null	Enforce required tags on resources



CHECKLIST

- Defined account strategy
- Implemented SCP guardrails
- Centralized logging & monitoring
- IAM Identity Center configured
- Automated provisioning with Control Tower / IaC

CROSS-REGION DISASTER RECOVERY

Build resilient systems that survive region failures and meet business continuity requirements.

WHY CROSS-REGION DR?

- ✓ Protect against region-wide outages
- ✓ Meet strict RTO/RPO business requirements
- ✓ Maintain data durability and availability
- ✓ Enable global active-active or active-passive
- ✓ Support compliance and data sovereignty

TYPICAL DR ARCHITECTURE (ACTIVE-PASSIVE EXAMPLE)



RTO vs RPO EXPLAINED



RTO (Recovery Time Objective)

Maximum acceptable time to restore service.
Example: RTO = 15 minutes



RPO (Recovery Point Objective)

Maximum acceptable data loss measured in time.
Example: RPO = 5 minutes

RTO/RPO STRATEGIES

STRATEGY	DESCRIPTION	RTO	RPO	USE CASES	COST
Backups & Restore	Restore from backups in DR region	Hours	Hours	Low critical apps, dev/test	\$
Pilot Light	Core infra running in DR region (small footprint)	10-60 min	5-15 min	Medium critical systems	\$\$
Warm Standby	Scaled-down environment running in DR region	Minutes	< 5 min	Business critical applications	\$\$\$
Multi-Site Active-Active	Active in multiple regions (synchronous/asynchronous)	Seconds	0	Global apps, very high availability	\$\$\$\$

KEY AWS SERVICES FOR DR

- Route 53 - DNS Failover
- AWS Backup - Centralized backups
- AWS Elastic Disaster Recovery (DRS) - Rapid recovery
- S3 Cross-Region Replication (CRR)
- RDS Cross-Region Read Replicas
- DynamoDB Global Tables
- CloudFormation StackSets
- AWS Organizations - Multi-account DR
- IAM Roles & Permissions
- CloudWatch Alarms & Events

DATA REPLICATION OPTIONS

S3 Cross-Region Replication Asynchronous replication of objects between regions.	EBS Snapshot Copy Copy snapshots to another region.
RDS Read Replicas Asynchronous replication for read scaling and DR.	EFS Backup & Replication AWS Backup or third-party replication.
DynamoDB Global Tables Multi-master replication across regions.	Database Logical Replication Use DMS for cross-region DB replication.
AWS Backup Cross-region copy of backup vaults.	Storage Gateway For hybrid DR to AWS.

DR AUTOMATION FLOW (HIGH LEVEL)



DR TESTING BEST PRACTICES

- ✓ Test regularly (quarterly or bi-annually).
- ✓ Use game days to simulate real failures.
- ✓ Validate RTO/RPO during each test.
- ✓ Automate as much as possible.
- ✓ Document runbooks clearly.
- ✓ Include rollback procedures.
- ✓ Review and improve after every test.

COMMON INTERVIEW QUESTIONS

- 1 How would you design a cross-region DR solution for a banking application?
- 2 What is the difference between RTO and RPO? How do they affect your design?
- 3 How would you achieve near-zero RPO for a critical database?
- 4 How would you test your DR plan without impacting production?
- 5 What are the trade-offs between active-active and active-passive?
- 6 How would you automate a DR failover in AWS?

DESIGN TRADE-OFFS



★ KEY TAKEAWAYS

- ✓ DR is about people, process, and technology.
- ✓ Design based on business impact, not services.
- ✓ Automate detection, failover and validation.
- ✓ Test regularly and learn continuously.
- ✓ Document everything like it will fail.

REAL WORLD DR SCENARIO (EXAMPLE)

Scenario:

Your e-commerce platform in us-east-1 experiences a region outage.

Goal:

Failover to us-west-2 within 15 minutes with less than 5 minutes of data loss.



METRICS TO TRACK

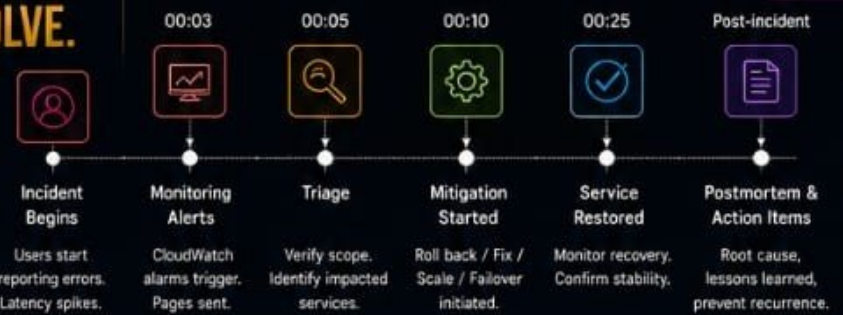
- ⌚ RTO Achieved vs Target
- 🕒 RPO Achieved vs Target
- ✓ Failover Success Rate
- 📈 Recovery Time Trend
- 📋 DR Test Pass Rate

PRODUCTION OUTAGES

DIAGNOSE. COMMUNICATE. RESOLVE.

Anyone can build. Senior engineers keep systems running when things break.

OUTAGE TIMELINE (EXAMPLE)



OUTAGES WILL HAPPEN

- Complex systems fail in complex ways.
- The goal is fast detection, clear communication, and quick recovery.
- Your response is what's being evaluated in senior interviews.

COMMON ROOT CAUSES

- Bad Deployments**
Bugs, config changes, incompatible DB migrations.
- Capacity Issues**
Traffic spikes, autoscaling misconfig, insufficient limits.
- Dependency Failures**
Downstream services, third-party APIs, network issues.
- Data Layer Issues**
DB overload, hot partitions, connection exhaustion.
- Infrastructure Failures**
AZ outage, hardware failure, network partitions.
- Cost Optimization Gone Wrong**
Overly aggressive savings leading to performance issues.

OUTAGE INVESTIGATION FRAMEWORK

- 1 DETECT**
Identify via alarms, logs, dashboards, or users.
Questions to ask:
• What is failing?
• When did it start?
• What is the impact?
- 2 SCOPE**
Determine blast radius and affected regions.
Questions to ask:
• Which users are impacted?
• Which services are impacted?
• Is it isolated or widespread?
- 3 HYPOTHEZIZE**
List possible causes based on symptoms.
Questions to ask:
• What changed recently?
• Any deployments or config changes?
• Are dependencies healthy?
- 4 INVESTIGATE**
Use metrics, logs, traces to confirm root cause.
Questions to ask:
• What do logs / metrics show?
• Where is latency / errors coming from?
• Can we reproduce?
- 5 MITIGATE**
Take action to restore service.
Questions to ask:
• What is the fastest safe fix?
• Do we need rollback or failover?
• How do we minimize impact?
- 6 VALIDATE**
Confirm recovery and monitor closely.
Questions to ask:
• Are error rates back to normal?
• Is performance stable?
• Any hidden impact?
- 7 POSTMORTEM**
Document, learn, and prevent recurrence.
Questions to ask:
• What was the root cause?
• Why didn't we catch it earlier?
• What will we do differently?

OUTAGE RESPONSE PLAYBOOK

- 1. Declare Incident**
Acknowledge and open incident channel.
- 2. Communicate Early**
Update stakeholders. Share facts, not assumptions.
- 3. Own the Problem**
One person drives the incident. No silos.
- 4. Mitigate Fast**
Restore service or reduce impact.
- 5. Communicate Updates**
Regular updates every 15-30 minutes.
- 6. Postmortem**
Blameless. Focus on systems, not people.

COMMUNICATION PRINCIPLES

- Be transparent.
- Be concise.
- Be factual.
- Avoid speculation.
- Communicate impact & ETA.
- Follow up when resolved.

REAL OUTAGE SCENARIO (EXAMPLE)



POSTMORTEM TEMPLATE

- Summary**
What happened in 1-2 sentences.
- Impact**
Who was impacted? For how long?
- Timeline**
Key events with timestamps.
- Root Cause**
What caused the outage?
- Contributing Factors**
Why did it happen?
- What Went Well**
What helped?
- What Didn't Go Well**
What slowed us down?
- Action Items**
What will we do to prevent this?

BEST PRACTICES

- Implement strong monitoring & alerting.
- Create and test runbooks regularly.
- Use autoscaling and health checks.
- Design for fault isolation.
- Maintain capacity headroom.
- Chaos test your systems.
- Review and improve after every incident.

KEY METRICS TO WATCH

- Error Rate (5xx %)
- Latency (p50, p90, p99)
- Traffic (Requests / RPS)
- CPU / Memory Utilization
- Database Connections
- Queue Depth / Backlog
- Dependency Health
- Saturation & Throttles

COMMON INTERVIEW QUESTIONS

- Tell me about a time you handled a production outage.
- How do you quickly identify the root cause of an outage?
- What tools do you use for monitoring and alerting?
- How do you communicate during an incident?
- How do you prevent the same outage from happening again?
- What's your incident response process?
- How do you test your systems for failure?
- How do you handle a situation where rollback is not possible?

REMEMBER

Senior engineers are judged not only on technical skills, but on how they behave under pressure.

Stay calm. Investigate. Communicate. Resolve. That's what leadership looks like.

COST OPTIMIZATION DECISIONS THAT MATTER

Optimize for value, not just for price. Senior engineers balance cost, performance, reliability, and agility.

COST OPTIMIZATION PRINCIPLES



Understand and allocate costs.



Right-size resources.



Use the right pricing model.



Eliminate waste continuously.



Design cost-aware architectures.

WHERE DOES AWS COST COME FROM?

	Compute (EC2, Lambda, ECS, EKS)	40–60%
	Storage (S3, EBS, EFS, Glacier)	15–25%
	Data Transfer	10–15%
	Databases (RDS, DynamoDB, ElastiCache)	10–20%
	Other Services (ALB, NAT, CloudWatch, etc.)	5–10%

PRICING MODELS – WHEN TO USE WHAT

MODEL	BEST FOR	COMMITMENT	DISCOUNT	FLEXIBILITY	EXAMPLE USE CASES
On-Demand	Short-term, unpredictable workloads	None	0%	★★★★★	Dev/Test, spiky traffic, unpredictable workloads
Savings Plans	Steady usage across instance families	1 or 3 years (\$/hour)	Up to 72%	★★★★☆	EC2, Fargate, Lambda steady state
Reserved Instances	Steady usage for specific instances	1 or 3 years (all upfront/partial)	Up to 72%	★★★☆☆	Databases, stateful workloads, long-term apps
Spot Instances	Fault-tolerant, interruptible workloads	None	Up to 90%	★☆☆☆☆	Batch jobs, CI/CD, ML training, data processing
Dedicated Hosts	Compliance, licensing, instance isolation	1 or 3 years	Variable	★★★★☆	BYOL, regulatory, per-socket licensing

COST OPTIMIZATION PILLARS

- Right-Sizing**
Match resources to actual usage.
- Resource Scheduling**
Stop what you don't use.
- Storage Optimization**
Choose the right storage tier & lifecycle.
- Data Transfer Optimization**
Keep data close, compress, cache.
- Architectural Efficiency**
Design for scale without over-provisioning.
- Continuous Monitoring**
Measure, alert, and improve.

RIGHT-SIZING EXAMPLES

SERVICE	OVER-PROVISIONED	RIGHT-SIZED APPROACH	POTENTIAL SAVINGS
EC2	m5.xlarge running at 10% CPU average	m6i.large or Graviton (t4g.large)	60–75%
RDS	db.r5.xlarge with low CPU & I/O	db.r6g.large with optimized throughput	50–70%
EBS	gp2 volumes not optimized	gp3 with right IOPS and throughput	20–60%
Lambda	Over-allocated memory (> 1024 MB)	Right-size memory based on duration	10–30%
EKS	Over-provisioned node groups	Cluster Autoscaler + Spot + Right-size	30–60%

COMMON COST ANTI-PATTERNS

- ✗ Running 24/7 non-production environments.
- ✗ Over-provisioned instances "just in case".
- ✗ Unused EBS volumes and snapshots.
- ✗ Unattached Elastic IPs.
- ✗ Data transfer across regions and AZs.
- ✗ Not using Reserved Instances or Savings Plans.
- ✗ Orphaned resources (left behind).

S3 COST OPTIMIZATION

- Use S3 Intelligent-Tiering for unknown access patterns.
- Use lifecycle policies to move data to IA/Glacier.
- Delete incomplete multipart uploads.
- Compress and deduplicate data before upload.
- Use S3 Transfer Acceleration only when needed.

S3 STORAGE CLASSES QUICK GUIDE

CLASS	USE CASE	RETRIEVAL	COST
S3 Standard	Frequently accessed data	Milliseconds	\$\$\$
S3 IA	Infrequent access	Milliseconds	\$\$
S3 One Zone-IA	Recreatable, infrequent	Milliseconds	\$
S3 Glacier Instant Retrieval	Archive, occasional access	Milliseconds	\$
S3 Glacier Flexible Retrieval	Archive	Minutes–Hours	\$
S3 Glacier Deep Archive	Long-term archive	Hours	¢

DATA TRANSFER COST TIPS

- Keep traffic within the same AZ or region.
- Use CloudFront to cache and distribute content.
- Compress data in transit and at rest.
- Use VPC Endpoints for AWS service traffic.
- Avoid NAT Gateways for high outbound traffic.

TOOLS TO MONITOR & OPTIMIZE

- AWS Cost Explorer**
Analyze trends and find savings opportunities.
- AWS Budgets**
Set budgets and get alerts.
- AWS Cost Anomaly Detection**
Detect unusual spend.
- Trusted Advisor**
Get recommendations.
- AWS Compute Optimizer**
Right-size your resources.
- CloudHealth / Cast.ai / Spot.io**
Advanced optimization platforms.

OPTIMIZATION DECISION FRAMEWORK



INTERVIEW SCENARIOS

- 1 Your monthly AWS bill is increasing by 40%. How do you investigate and reduce costs?
- 2 How would you reduce costs for a multi-region application?
- 3 How would you optimize costs for a data lake on S3?
- 4 How do you choose between Savings Plans and Reserved Instances?
- 5 How would you reduce NAT Gateway costs?
- 6 How would you optimize costs without affecting performance or reliability?

★ KEY TAKEAWAYS

- ✓ Cost optimization is a continuous process.
- ✓ Right-size, right-time, right-price.
- ✓ Optimize architectures, not just resources.
- ✓ Use data to drive every decision.
- ✓ Small savings per service = big impact.



PRO TIP

The best architects save money without compromising on customer experience, reliability, or speed.

COST OPTIMIZATION CHECKLIST

- ✓ Review and tag all resources.
- ✓ Set budgets and alerts.
- ✓ Right-size resources.
- ✓ Use Savings Plans / RIs where possible.
- ✓ Optimize storage and lifecycle.
- ✓ Audit idle and orphaned resources.
- ✓ Monitor & iterate continuously.

“The art of optimization is maximizing value delivered per dollar spent.”

PERFORMANCE & SCALABILITY AT SCALE

Design for millions. Scale with confidence.
Measure, identify, and remove bottlenecks.

SCALABILITY PRINCIPLES

- ✓ Scale horizontally, not vertically.
- ✓ Stateless wherever possible.
- ✓ Decouple components.
- ✓ Automate scaling based on metrics.
- ✓ Design for failure and graceful degradation.

SCALING PATTERNS

HORIZONTAL SCALING



Add more instances behind a load balancer. Best for most workloads.

AUTO SCALING GROUP



Automatically add/remove instances based on demand and policies.

ELASTIC LOAD BALANCING



Distribute traffic across multiple AZs and instances for high availability.

SERVERLESS SCALING



Scale automatically from zero to millions. Pay per request.

DATABASE SCALING



Read replicas, sharding, caching, and partitioning.

PAGE
9

PERFORMANCE BEST PRACTICES

- Use CloudFront for caching and global delivery.
- Enable caching (ElastiCache, DAX, Redis, Memcached).
- Optimize database queries and add proper indexes.
- Use async processing (SQS, SNS, Lambda).
- Compress data in transit and at rest.
- Right-size resources and choose correct instance types.

PERFORMANCE TROUBLESHOOTING FRAMEWORK

	DEFINE THE PROBLEM	WHAT TO CHECK	TOOLS TO USE
1	Understand the impact, scope and symptoms.	- What is slow? - Who is affected? - When did it start? - What changed?	CloudWatch Alarms & Metrics
2	COLLECT DATA Gather metrics, logs, and traces.	- CPU, Memory, Network - Application logs - Latency percentiles - Error rates	CloudWatch Logs X-Ray Traces
3	IDENTIFY BOTTLENECK Find the component causing the issue.	- High CPU / Memory? - Slow DB queries? - Network latency? - Lock contention?	CloudWatch Insights Performance Insights
4	ANALYZE ROOT CAUSE Deep dive to find the real issue.	- Inefficient code / queries? - Resource limits? - Traffic spike? - External dependencies?	X-Ray Application Signals
5	MITIGATE & RESOLVE Apply fixes and restore service.	- Scale resources - Optimize code / queries - Enable caching - Fix configuration	Auto Scaling Parameter Store
6	VALIDATE & IMPROVE Ensure stability and prevent recurrence.	- Monitor key metrics - Set alarms & dashboards - Run load tests - Document learnings	CloudWatch Dashboards

COMMON BOTTLENECKS & FIXES

BOTTLENECK	HOW TO FIX
High CPU	- Scale out - Optimize code - Use Graviton / better instance types
High Memory	- Add more memory - Optimize app - Use cache
DB Slow Queries	- Add indexes - Optimize queries - Read replicas - Caching (DAX/Redis)
Network Latency	- Use CloudFront - Place resources closer (AZ/Region) - Reduce hops
High I/O	- Use SSD (gp3/io2) - Increase IOPS - Cache frequently accessed data
Throttling	- Increase limits - Use backoff & retry - Distribute load

LOAD TESTING STRATEGY

Define Goals	What are you testing? (Throughput, latency, concurrency, etc.)	TOOLS TO USE - AWS Load Generator - JMeter - Gatling - k6 - Locust - Artillery
Create Test Data	Use realistic data and workflows.	
Ramp Up Load	Start low and increase gradually.	
Monitor & Measure	Use metrics, logs and traces.	
Analyze Results	Find bottlenecks and failure points.	
Optimize & Repeat	Improve and test again.	

CACHING STRATEGIES

CloudFront	Cache static and dynamic content at the edge.
ElastiCache (Redis/Memcached)	Cache frequently accessed data in memory.
DynamoDB DAX	Microsecond read caching for DynamoDB.
Application Cache	Cache at application layer (in-memory).
Browser Cache	Cache static assets in user's browser.

BEST PRACTICES

- Set proper TTLs.
- Invalidate cache on updates.
- Cache at multiple layers.
- Monitor cache hit ratios.
- Don't cache sensitive data.

HIT RATIO
92%

KEY PERFORMANCE METRICS

Latency (p50, p90, p99)	Lower is better.
Throughput (RPS / TPS)	More is better.
Error Rate (%)	Lower is better.
Availability (%)	Higher is better.
Saturation	How close to max capacity.
Cost per Request	Optimize for efficiency.

REAL WORLD SCENARIO (EXAMPLE)



KEY TAKEAWAYS

- ✓ Performance is a feature, not an afterthought.
- ✓ Measure everything you can.
- ✓ Scale horizontally and automate.
- ✓ Cache aggressively and smartly.
- ✓ Optimize continuously based on data.
- ✓ Test under real-world conditions.

SECURITY & COMPLIANCE AT ENTERPRISE SCALE

PAGE
10

Build secure-by-design architectures that protect data, identities, and workloads.

SECURITY PRINCIPLES

- ✓ Least privilege access.
- ✓ Defense in depth.
- ✓ Fail secure by default.
- ✓ Encrypt everything.
- ✓ Monitor, detect, and respond.
- ✓ Automate security and compliance.

SHARED RESPONSIBILITY MODEL



WELL-ARCHITECTED SECURITY PILLARS

- Protect identities
- Apply least privilege
- Encrypt data in transit and at rest
- Enable traceability
- Automate compliance
- Prepare for incident response

AWS IAM BEST PRACTICES

- ✓ Use IAM Identity Center (SSO) for users.
- ✓ Use groups, not individual permissions.
- ✓ Apply least privilege policies.
- ✓ Use MFA for all users, especially admins.
- ✓ Rotate access keys and use short-lived creds.
- ✓ Use IAM roles for EC2, Lambda, and services.
- ✓ Monitor with CloudTrail & Access Analyzer.

ACCESS CONTROL STRATEGY



NETWORK SECURITY

- Use VPC with private subnets.
- Use Security Groups (stateful).
- Use NACLs (stateless) at subnet level.
- Use AWS WAF for web protection.
- Use AWS Shield for DDoS protection.
- Use VPC Endpoints to keep traffic on AWS network.

DATA PROTECTION

- Encrypt in transit: TLS/HTTPS.
- Encrypt at rest: KMS (SSE-KMS).
- Use customer-managed KMS keys.
- Use Secrets Manager for secrets.
- Enable S3 Block Public Access.
- Classify data: Public / Internal / Confidential / Restricted.

AWS SECURITY SERVICES OVERVIEW

IAM	Manage identities and access securely.	Security Hub	Centralized security findings & compliance.
AWS SSO	Centralize access for users & accounts.	Config	Track resource changes & compliance.
KMS	Manage encryption keys and data protection.	CloudTrail	Audit API activity and events.
Secrets Manager	Store and rotate secrets securely.	Macie	Find and protect sensitive data.
GuardDuty	Intelligent threat detection using ML.	Detective	Investigate and analyze security issues.
Inspector	Automated vulnerability assessment.	Firewall Manager	Centrally manage security policies.

COMPLIANCE & GOVERNANCE

- Use AWS Organizations for multi-account governance.
- Apply Service Control Policies (SCPs) to guard accounts.
- Use AWS Config rules for compliance checks.
- Enable AWS Audit Manager for continuous compliance.
- Map to frameworks: ISO 27001, SOC 2, PCI DSS, HIPAA.
- Use tag policies for standardization and control.
- Regularly review access with IAM Access Analyzer.

SECURITY INCIDENT RESPONSE FLOW



TOOLS: GuardDuty • Security Hub • CloudTrail • VPC Flow Logs • Detective

ENCRYPTION OPTIONS

SERVICE	IN TRANSIT	AT REST	KEY MANAGEMENT
S3	HTTPS / TLS	SSE-S3 / SSE-KMS	AWS KMS / CMK
EBS	N/A	SSE-KMS	AWS KMS / CMK
RDS	TLS	SSE-KMS	AWS KMS / CMK
DynamoDB	TLS	SSE-KMS	AWS KMS / CMK
EFS	TLS	SSE-KMS	AWS KMS / CMK

COMMON INTERVIEW QUESTIONS

- How do you implement least privilege access in AWS?
- How would you secure an S3 bucket handling sensitive data?
- How do you monitor and detect security threats in AWS?
- How would you respond to a compromised IAM user?
- What is the difference between Security Groups and NACLs?
- How would you ensure compliance across multiple accounts?
- How do you protect data in transit and at rest?
- How would you secure a serverless application?

REAL WORLD SCENARIO (EXAMPLE)

Scenario: An unusual spike in API calls triggered errors in production.



Goal: Detect early, contain fast, recover safely, and prevent recurrence.

KEY TAKEAWAYS

- ✓ Security is everyone's responsibility.
- ✓ Least privilege and encryption are non-negotiable.
- ✓ Automate security, monitoring and compliance.
- ✓ Prepare for incidents before they happen.
- ✓ Continuous monitoring = continuous protection.

CASE STUDY 1: BANKING SYSTEM ARCHITECTURE ON AWS

Design a secure, highly available, and scalable banking platform to handle millions of transactions daily.

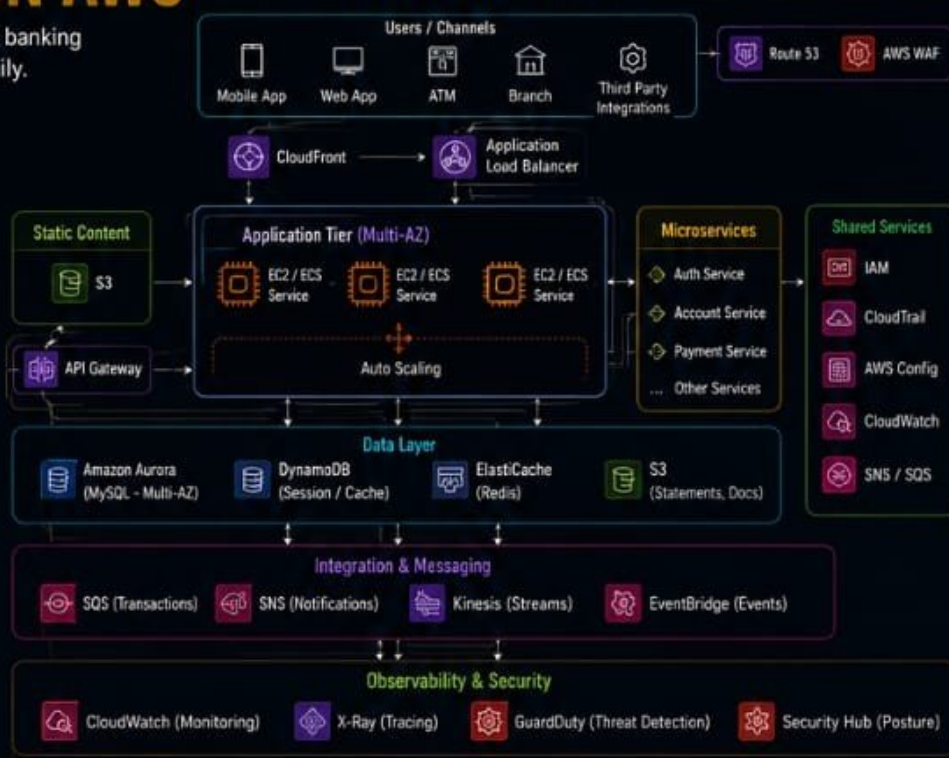
KEY REQUIREMENTS

- ✓ High availability (99.99%+)
- ✓ Strong consistency for transactions
- ✓ Low latency for real-time operations
- ✓ Data security & regulatory compliance
- ✓ Audit logging and traceability
- ✓ Scalability for peak loads
- ✓ Disaster recovery (RPO < 15 min, RTO < 1 hr)

DATA FLOW

- 1 User requests via app/web/ATM.
- 2 Route 53 resolves DNS and directs to CloudFront / ALB.
- 3 Requests hit microservices running on EC2/ECS.
- 4 Services read/write data from Aurora or DynamoDB.
- 5 Events published to SQS / EventBridge for async processing.
- 6 Notifications sent via SNS.
- 7 Logs, metrics, and traces captured for monitoring and audit.

HIGH-LEVEL ARCHITECTURE



CORE SERVICES & PURPOSE

SERVICE	PURPOSE
Route 53	DNS routing with health checks
CloudFront	Global edge caching, SSL termination
ALB	Distribute traffic across app instances
EC2 / ECS	Run microservices (stateless)
Aurora (Multi-AZ)	Primary transactional database
DynamoDB	Sessions, user preferences, idempotency
ElastiCache (Redis)	Caching, tokens, rate limiting
SQS / SNS	Async transactions, notifications
S3	Documents, statements, backups
Kinesis	Real-time transaction streaming
CloudWatch & X-Ray	Monitoring, tracing, and observability

SECURITY & COMPLIANCE

- ✓ All traffic via HTTPS (TLS 1.2+).
- ✓ Data encrypted at rest (KMS) and in transit.
- ✓ IAM least privilege & MFA for users.
- ✓ Secrets stored in AWS Secrets Manager.
- ✓ VPC with private subnets for data tier.
- ✓ WAF for DDoS and OWASP protection.
- ✓ CloudTrail for audit logging.
- ✓ Comply with PCI DSS, GDPR, SOX.

COMPLIANCE MAPPING (EXAMPLE)

REGULATION	AWS SERVICE
PCI DSS	AWS WAF, Secrets Manager, KMS, CloudTrail
GDPR	S3 (Data residency), Macie, KMS
SOX	CloudTrail, AWS Config, GuardDuty

AVAILABILITY & RESILIENCE

- ✓ Multi-AZ for all critical components.
- ✓ Aurora Multi-AZ with automatic failover.
- ✓ Auto Scaling for app tier.
- ✓ SQS for decoupling and retry.
- ✓ Backups automated (RDS, EBS, S3).
- ✓ Health checks + Route 53 failover.
- ✓ Disaster Recovery in another region.

DR STRATEGY

COMPONENT	STRATEGY
Aurora	Cross-Region Read Replica
S3	Cross-Region Replication
DynamoDB	Global Tables
Route 53	Failover Routing Policy
RPO / RTO	< 15 min / < 1 hr

NON-FUNCTIONAL DESIGN DECISIONS

Consistency	Strong consistency for transactions (Aurora). Eventual consistency for analytics.
Scalability	Horizontal scaling for stateless services. Serverless for event-driven workloads.
Latency	Global CDN (CloudFront) & Edge caching. ElastiCache for sub-millisecond reads.
Cost Optimization	Use Savings Plans for EC2, S3 Intelligent-Tiering. Aurora Serverless v2 for variable workloads.
Operations	Infrastructure as Code (Terraform / CDK). CI/CD with blue/green or canary deployments.

POSSIBLE TRADE-OFFS

Strong Consistency vs Scalability	Strong consistency ensures accuracy but limits horizontal write scaling.
Synchronous vs Asynchronous	Synchronous gives immediate feedback but increases latency.
Multi-AZ vs Cost	High availability increases cost (significant uplift).
Caching vs Freshness	Caching improves latency but can serve stale data.
Security vs Speed	More security layers add latency (overhead).

INTERVIEWER EXPECTATIONS

- ✓ Clarify requirements before designing.
- ✓ Explain API design and data modeling.
- ✓ Show how you handle transactions (ACID vs BASE).
- ✓ Discuss scaling, monitoring, and failure scenarios.
- ✓ Justify AWS service choices and trade-offs.
- ✓ Think about security, compliance, and auditability.

EXAMPLE METRICS TO MONITOR

Transaction latency (p95, p99)	Queue depth (SQS)
Error rate (%)	CPU / Memory utilization
Throughput (TPS)	Login failures
Database connections	DDoS attempts blocked
Cache hit ratio	Backup success rate

INTERVIEW QUESTIONS (EXAMPLE)

- 1 How would you handle a sudden traffic spike?
- 2 How do you ensure zero data loss during a failure?
- 3 How would you design idempotent transactions?
- 4 What strategies would you use for fraud detection?
- 5 How would you migrate this system to another region?

Pro Tip

Always start with requirements, think in components, justify your choices, and discuss trade-offs like a senior engineer.

CASE STUDY 2: GLOBAL E-COMMERCE PLATFORM ON AWS

Design a global e-commerce platform that handles millions of users, high traffic during sales events, secure payments, inventory management, and fast delivery experience.

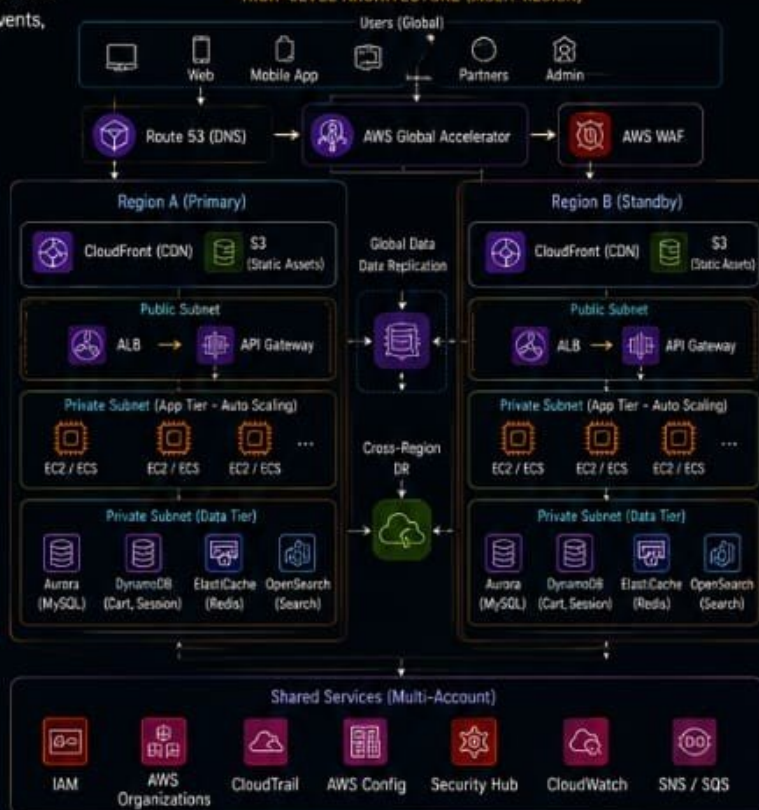
KEY REQUIREMENTS

- ✓ Global user base with low latency
- ✓ Handle flash sales & high concurrency
- ✓ Secure payments & PCI DSS compliance
- ✓ Inventory, order & shipment tracking
- ✓ Search, recommendations & reviews
- ✓ High availability (99.99%+)
- ✓ Scalable & cost optimized
- ✓ Disaster recovery across regions

BUSINESS FEATURES

- 👤 User Accounts & Profiles
- 📦 Product Catalog & Search
- 🛒 Shopping Cart & Checkout
- 💳 Payments & Refunds
- 📦 Order Management
- 📦 Inventory & Warehouse
- 📦 Shipping & Tracking
- ⭐ Reviews & Recommendations
- 🎁 Promotions & Coupons
- 📊 Analytics & Reporting

HIGH-LEVEL ARCHITECTURE (MULTI-REGION)



DATA FLOW OVERVIEW

- 1 User requests are routed via Route 53 + Global Accelerator.
- 2 CloudFront serves static content from edge locations.
- 3 Dynamic requests go to ALB via API Gateway.
- 4 App tier (EC2/ECS) processes requests and calls microservices.
- 5 Data stored in Aurora, DynamoDB or cached in ElastiCache.
- 6 Search queries served by OpenSearch Service.
- 7 Events (orders, payments) published to SQS/SNS.
- 8 Analytics data streamed to Kinesis and processed.

TECHNOLOGIES USED

Compute : EC2 / ECS / Fargate
Networking : VPC, ALB, API GW, CloudFront
Database : Aurora MySQL, DynamoDB
Caching : ElastiCache (Redis)
Search : OpenSearch Service
Messaging : SQS, SNS, EventBridge
Storage : S3, EBS, EFS
Analytics : Kinesis, Glue, Athena, QuickSight
Security : IAM, WAF, Shield, KMS, Secrets Manager
Monitoring : CloudWatch, X-Ray, CloudTrail

MICROSERVICES BREAKDOWN

SERVICE	RESPONSIBILITY
User Service	Authentication, profiles, addresses
Catalog Service	Products, categories, pricing
Cart Service	Manage cart, coupons, sessions
Order Service	Orders, cancellation, returns
Payment Service	Payments, refunds, fraud checks
Inventory Service	Stock, warehouses, reservations
Shipping Service	Carriers, rates, tracking
Review Service	Reviews, ratings
Notification Service	Emails, SMS, in-app alerts

DATABASE STRATEGY

Aurora MySQL (Primary DB)	• User data, orders, payments (ACID) • Multi-AZ for high availability
DynamoDB	• Cart, sessions, wishlists • High throughput, low latency
ElastiCache Redis	• Product cache, sessions, tokens • Improve read performance
OpenSearch	• Product search, autocomplete • Full-text search & analytics
S3	• Static assets, invoices, receipts • Versioning & lifecycle policies

SCALABILITY STRATEGY

- ✓ Auto Scaling for all stateless services (EC2/ECS).
- ✓ CloudFront CDN for global content delivery.
- ✓ DynamoDB On-Demand for variable workloads.
- ✓ Read replicas for Aurora to handle read scale.
- ✓ ElastiCache to offload database.
- ✓ SQS decouples services & absorbs spikes.
- ✓ Partitioning & sharding for hot tables.
- ✓ Horizontal scaling for microservices.

SECURITY & COMPLIANCE

- ✓ IAM least privilege & MFA enforcement.
- ✓ WAF for SQLi, XSS, bot protection.
- ✓ AWS Shield Advanced for DDoS.
- ✓ KMS for encryption at rest.
- ✓ TLS 1.2+ for data in transit.
- ✓ PCI DSS Level 1 compliance.
- ✓ Secrets Manager for credentials.
- ✓ CloudTrail for audit logs.
- ✓ VPC with private subnets & NAT.

PAYMENT ARCHITECTURE



OBSERVABILITY & MONITORING

- 📊 CloudWatch (Logs, Metrics, Alarms)
- 🔍 X-Ray (Distributed Tracing)
- 📋 CloudTrail (Audit Logs)
- 🔔 Alarms to SNS / PagerDuty.
- 📊 Dashboards for key business metrics.
- 🔗 Trace user requests across microservices.

DISASTER RECOVERY PLAN

ASPECT	STRATEGY
RPO	< 15 minutes
RTO	< 1 hour
Backup	Aurora automated backups + S3 cross-region replication
Failover	Route 53 health checks failover to standby region
Testing	DR drills every quarter

COST OPTIMIZATION

- ✓ Use Savings Plans for EC2 & Fargate.
- ✓ S3 Intelligent-Tiering for old files.
- ✓ CloudFront caching to reduce origin requests.
- ✓ Right-size instances & use Graviton.
- ✓ DynamoDB On-Demand for predictable cost.
- ✓ Lifecycle policies for logs & backups.

TRADE-OFFS

CHOICE	TRADE-OFF
Aurora vs DynamoDB	Aurora for relational/ACID, DynamoDB for low-latency scale.
Monolith vs Microservices	Microservices improve scale but increase operational complexity.
Multi-region (Active-Active)	Lower latency but higher cost and data consistency challenges.
Caching	Improves performance but risk of stale data.

INTERVIEW QUESTIONS (EXAMPLE)

- 1 How would you handle a flash sale with 10x traffic spike?
- 2 How do you ensure idempotent order processing?
- 3 How would you design the inventory reservation system?
- 4 What is your strategy for global expansion?
- 5 How do you handle payment failures?
- 6 How would you optimize search performance?

KEY TAKEAWAYS

- ✓ Design for scale, availability and security.
- ✓ Decouple services with messaging.
- ✓ Cache aggressively & replicate globally.
- ✓ Automate deployment & monitoring.
- ✓ Always test for failure and recovery.



Pro Tip
Always validate your design with real-world constraints: traffic, cost, and operations.

CASE STUDY 3: SOCIAL MEDIA FEED & ENGAGEMENT PLATFORM

PAGE
13

Design a social media platform that supports user posts, news feed, likes, comments, follows, real-time notifications, and media uploads at massive scale.

KEY REQUIREMENTS

- ✓ Hundreds of millions of users
- ✓ News feed generation in < 200ms
- ✓ Like, comment, share & follow
- ✓ Real-time notifications
- ✓ Media upload & delivery
- ✓ High read/write throughput
- ✓ Strong consistency for critical actions
- ✓ High availability & fault tolerance
- ✓ Scalable globally with low latency

CORE FEATURES

- 👤 User authentication & profiles
- 📰 News feed (home feed)
- ✍️ Create, edit, delete posts
- 👍 Like, comment, share posts
- 👤 Follow / unfollow users
- 🔔 Real-time notifications
- 💬 Direct messaging (future)
- 🔍 Search users & posts
- 📶 Media upload & delivery
- 🛠️ Admin dashboard & moderation

HIGH-LEVEL ARCHITECTURE



DATA FLOW OVERVIEW

- 1 User authenticates via User Service.
- 2 User creates a post (text / media).
- 3 Post Service stores metadata in DynamoDB and publishes event to Kinesis.
- 4 Feed Service fan-out writes to followers' feed (DynamoDB).
- 5 Users request feed → read from DynamoDB + cache.
- 6 Likes/Comments stored in DynamoDB and cached in Redis.
- 7 Real-time notifications sent via WebSocket / SNS.
- 8 Media uploaded to S3, processed, delivered via CloudFront.
- 9 Search queries served by OpenSearch Service.

TECHNOLOGIES USED

Compute : ECS / EKS, Lambda
Storage : S3, DynamoDB, ElastiCache
Database : DynamoDB, OpenSearch
Messaging : SNS, SQS, Kinesis, EventBridge
API : API Gateway, AppSync (GraphQL)
Network : CloudFront, Route 53, WAF
Security : IAM, Cognito, Shield, KMS
Monitoring : CloudWatch, X-Ray, CloudTrail

FEED DESIGN OPTIONS

APPROACH	DESCRIPTION	PROS	CONS
Fan-out on Write	Write posts to followers' feeds	Fast reads	High write amplification
Fan-out on Read	Generate feed when user reads	Lower writes	Slower reads
Hybrid Approach	Pre-compute for active users, on-demand for others	Balanced	Complex logic

SCALABILITY STRATEGY

- ✓ Microservices deployed on ECS / EKS with Auto Scaling.
- ✓ DynamoDB auto scaling with on-demand capacity.
- ✓ CloudFront caching to offload read traffic.
- ✓ ElastiCache Redis for sessions, timelines, counters.
- ✓ Kinesis for real-time events and analytics.
- ✓ S3 + CloudFront for media delivery.

CONSISTENCY MODEL

ENTITY	CONSISTENCY	STRATEGY
User Auth	Strong	DynamoDB (Strong Reads)
Post Write	Strong	DynamoDB (Conditional Writes)
News Feed	Eventual	DynamoDB + Caching
Like/Comment	Eventual	DynamoDB + Redis
Notifications	Eventual	SNS / SQS / WebSocket

SECURITY & COMPLIANCE

- ✓ Cognito for AuthN / AuthZ.
- ✓ IAM least privilege access.
- ✓ WAF for malicious request blocking.
- ✓ KMS encryption for data at rest.
- ✓ TLS 1.2+ for data in transit.
- ✓ CloudTrail for audit logging.
- ✓ Comply with GDPR / CCPA.

DATABASE DESIGN (EXAMPLE)

TABLE / STORE	PURPOSE	KEY DESIGN (EXAMPLE)
Users	Store user profiles	PK: userId
Posts	Store posts	PK: postId, SK: createdAt
Feed	User timelines	PK: userId, SK: timestamp (desc)
Likes	Track likes	PK: postId, SK: userId
Comments	Store comments	PK: postId, SK: createdAt
Follows	Social graph	PK: followerId, SK: followeeId

SOCIAL GRAPH MODEL (FOLLOW)

DynamoDB Table: Follows			
Followers of User A (who follows A)		Following by User A (A follows)	
PK (followerId)	SK (followeeId)	PK (followerId)	SK (followeeId)
User1	A	A	UserX
User2	A	A	UserY
User3	A	A	UserZ
...	...	...	...

Allows bi-directional queries.

NOTIFICATION ARCHITECTURE



REAL-TIME FEED STRATEGY



OBSERVABILITY & MONITORING

- 📊 CloudWatch Dashboards: latency, traffic, errors.
- 🚨 Alarms: high latency, 5xx errors, throttles.
- 🔍 X-Ray: trace requests across microservices.
- 📝 CloudWatch Logs Insights: analyze logs.
- 📡 Kinesis + S3: store logs for analytics.

DR & BACKUP STRATEGY

- ✓ Multi-AZ for all services.
- ✓ DynamoDB PITR enabled.
- ✓ S3 cross-region replication.
- ✓ Automated backups of OpenSearch.
- ✓ Route 53 health checks + failover.
- ✓ RPO < 15 min, RTO < 1 hr.

INTERVIEW QUESTIONS (EXAMPLE)

- 1 How would you design the news feed system?
- 2 Fan-out on write vs fan-out on read?
- 3 How would you handle real-time notifications?
- 4 How would you scale the system to billions of users?
- 5 How would you handle media uploads at scale?
- 6 How would you ensure data consistency?

KEY TAKEAWAYS

- ✓ Break down social features into microservices.
- ✓ Use event-driven architecture for scalability.
- ✓ Optimize for reads (caching, CDN, data modeling).
- ✓ Design for real-time and near-real-time operations.
- ✓ Plan for growth, availability, and global scale.



PRO TIP

In social platforms, the feed is the product. Invest in data modeling and caching strategies to deliver fast, consistent, and engaging user experiences.

METRICS TO TRACK

- 📊 Feed latency (p50, p95, p99)
- 👤 DAU / MAU
- 📄 Posts per second
- 📈 Engagement rate (likes, comments, shares)
- 📬 Notification delivery rate
- 🔍 Cache hit ratio
- 📉 Error rate (%)
- 📈 Storage growth rate

CASE STUDY 4: SAAS

MULTI-TENANT PLATFORM ON AWS

PAGE
14

Design a secure, scalable, and cost-efficient SaaS platform that supports thousands of tenants with data isolation, customization, billing, and high availability.

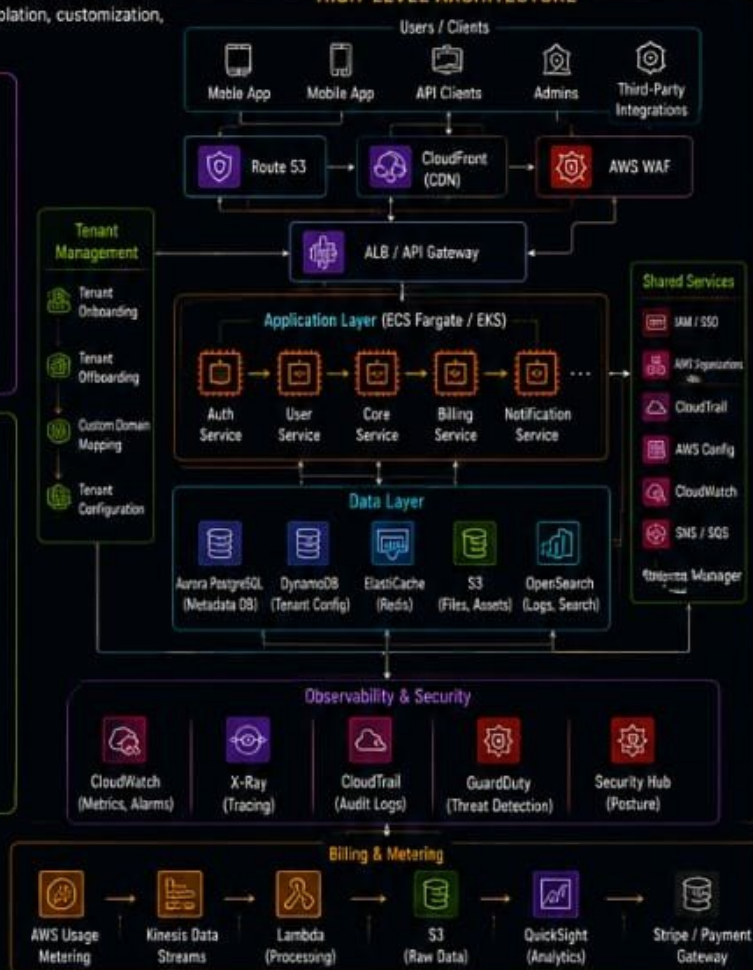
KEY REQUIREMENTS

- Multi-tenancy with strong isolation
- Tenant onboarding & offboarding
- Role-based access control (RBAC)
- Usage metering & billing
- Custom domains per tenant
- High availability (99.99%+)
- Scalable to thousands of tenants
- Secure by design & compliant

SAAS CORE FEATURES

- Tenant Registration & Onboarding
- Tenant Isolation (Data & Access)
- Subscription & Billing Management
- Usage Metering & Quotas
- Custom Domains & Branding
- Centralized Admin Dashboard
- Audit Logs & Compliance
- Notifications & Integrations
- API for Extensibility

HIGH-LEVEL ARCHITECTURE



DATA FLOW OVERVIEW

- Users access the application via custom domain or subdomain.
- Requests are routed through Route 53, CloudFront, and WAF.
- Traffic hits ALB / API Gateway.
- Microservices process the request in a tenant-aware context.
- Data is read/written to tenant-appropriate storage.
- Events sent to SNS / SQS for async processing.
- Logs, metrics, and traces captured for monitoring.
- Usage metered and sent to billing pipeline for invoicing.

TECHNOLOGIES USED

- Compute:** ECS Fargate / EKS
- API:** API Gateway
- Database:** Aurora PostgreSQL, DynamoDB
- Cache:** ElastiCache (Redis)
- Storage:** S3, EBS, EFS
- Search:** OpenSearch Service
- Messaging:** SNS, SQS, EventBridge, Kinesis
- Identity:** IAM, Cognito, SSO
- Monitoring:** CloudWatch, X-Ray, CloudTrail
- Security:** WAF, Shield, GuardDuty, Security Hub

MULTI-TENANCY MODELS

MODEL	DESCRIPTION	BEST FOR
Shared Database Shared Schema	All tenants share same database and schema.	Early stage, low isolation needs
Shared DB Separate Schema	Each tenant has separate schema.	Medium scale, better isolation
Separate Database Per Tenant	Each tenant has its own database.	High isolation, compliance
Hybrid Model	Critical tenants get separate DB, others share.	Large scale, cost optimized

TENANT ISOLATION STRATEGY

- Access Isolation:** Tenant context in token (JWT) & enforced in application layer.
- Data Isolation:** Row-level security / separate schema / separate database.
- Network Isolation:** Private subnets, SGs, and NACLs.
- Custom Domain Isolation:** Tenant-specific domain mapped via CloudFront + ACM.

SCALABILITY STRATEGY

- Auto Scaling for ECS / EKS services.
- Aurora read replicas for read scaling.
- DynamoDB On-Demand for bursty loads.
- CloudFront CDN for global users.
- Cache frequently accessed data in Redis.
- S3 for static files and media.

SECURITY & COMPLIANCE

- Tenant-aware RBAC with least privilege.
- Encrypt data at rest (KMS) and in transit (TLS).
- Secrets stored in AWS Secrets Manager.
- Audit all actions with CloudTrail.
- Comply with GDPR, SOC 2, ISO 27001.
- Regular vulnerability scans & patching.
- WAF + Shield for DDoS protection.

SUBSCRIPTION & BILLING FLOW



TENANT LIFECYCLE



COST OPTIMIZATION

- Use Fargate Spot for batch jobs.
- Leverage Savings Plans & Reserved Instances.
- Use Intelligent services for infrequent data.
- Auto Scaling to scale in during off-peak.
- Right-size Aurora & ElastiCache instances.
- Use CloudFront to reduce data transfer costs.

EXAMPLE USE CASES

- Project Management SaaS
- CRM / Customer Support
- HR & Payroll Systems
- Accounting & Finance Apps
- Collaboration Platforms
- Learning Management Systems

INTERVIEW QUESTIONS (EXAMPLE)

- How would you design a multi-tenant SaaS platform?
- How do you ensure tenant data isolation?
- How do you handle tenant onboarding and offboarding?
- How would you implement usage metering and billing?
- How do you handle custom domains for tenants?
- How do you achieve high availability and scalability?

KEY TAKEAWAYS

- Start simple, design for growth.
- Tenant isolation is critical for security & trust.
- Use managed services to reduce operational overhead.
- Automate onboarding, billing, and monitoring.
- Security, compliance, and observability are non-negotiable.

PRO TIP
Design for automation and self-service.
That's what makes a SaaS platform truly scalable.

CASE STUDY 5: LIVE STREAMING PLATFORM ON AWS

PAGE
15

Design a global live streaming platform that supports millions of concurrent viewers, low latency, adaptive bitrate streaming, chat, monetization, and high availability.

HIGH-LEVEL ARCHITECTURE



KEY REQUIREMENTS

- Support millions of concurrent viewers
- Low latency (< 10s glass-to-glass)
- Adaptive bitrate streaming (HLS/DASH)
- Global reach with minimal buffering
- Real-time chat and reactions
- Secure content (DRM, token authentication)
- Monetization (subscriptions, donations)
- High availability (99.99%+) & scalable

PLATFORM FEATURES

- Live streaming (low latency & high quality)
- Adaptive bitrate streaming (HLS/DASH)
- Global content delivery (CDN)
- Real-time chat, reactions, and gifting
- Monetization (subscriptions, donations)
- Recording & on-demand playback
- Content protection (DRM, tokens)
- Creator dashboard & analytics
- Search, recommendations & discovery

DATA FLOW OVERVIEW

- 1 Streamer sends live video via RTMP/SRT to MediaLive.
- 2 MediaLive ingests and forwards to MediaConvert for transcoding.
- 3 MediaPackage packages the content into HLS/DASH formats.
- 4 Packaged content stored in S3 (origin storage).
- 5 CloudFront distributes globally to viewers with edge caching.
- 6 Viewers playback via HLS/DASH with adaptive bitrate.
- 7 Real-time chat and reactions handled via IVS Chat + APIs.
- 8 Metadata, user data, and chat messages stored in databases.
- 9 Monitoring, alerts, and security ensure reliability and safety.

TECHNOLOGIES USED

- Compute:** MediaLive, MediaConvert, Lambda
- Networking:** Route 53, CloudFront, WAF, ALB
- Storage:** S3, EBS
- Database:** DynamoDB, Aurora
- Cache:** ElastiCache (Redis)
- Messaging:** Amazon SNS, SQS, IVS Chat
- Analytics:** Kinesis, QuickSight
- Security:** IAM, Shield, GuardDuty, KMS
- Monitoring:** CloudWatch, X-Ray, CloudTrail

STREAMING PROTOCOLS

PROTOCOL	USE CASE	LATENCY	NOTES
RTMP	Ingest (Broadcasters)	1-3s	Widely used
SRT	Ingest (Low Loss)	< 1s	Secure, Reliable
HLS	Playback	10-30s	Widely supported
DASH	Playback	10-30s	Adaptive, Standard
WebRTC	Ultra Low Latency	< 1s	Browser based

SCALABILITY STRATEGY

- Auto Scaling for MediaConvert workers.
- MediaPackage origin endpoints scale automatically.
- CloudFront scales globally with edge caching.
- S3 + Multi-AZ for high durability and availability.
- DynamoDB On-Demand for spiky workloads.
- ElastiCache for real-time chat & presence.

LOW LATENCY APPROACH

- Use SRT for ingest.
- Transcode with optimized settings.
- Use Low-Latency HLS (LL-HLS) / CMAF.
- Edge caching with CloudFront.
- Keep origin close to viewers.
- Optimize network path & peering.

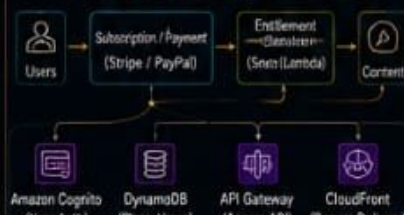
SECURITY & COMPLIANCE

- IAM least privilege access.
- CloudFront + WAF for protection.
- DRM (Widevine, FairPlay) for content.
- Token-based authentication.
- Encrypt data in transit (TLS 1.2+).
- Encrypt data at rest (KMS).
- Audit with CloudTrail & Config.

DRM & CONTENT PROTECTION



MONETIZATION ARCHITECTURE



ON-DEMAND & RECORDING



GLOBAL ARCHITECTURE



DATA STORES

SERVICE	PURPOSE
DynamoDB	User profiles, streams, chat metadata
Aurora	User accounts, payments, billing
ElastiCache	Real-time chat, presence, leaderboards
S3	Videos, thumbnails, recordings, logs
OpenSearch	Search, recommendations, analytics

OBSERVABILITY & MONITORING

COMPONENT	DR STRATEGY
CloudWatch	Metrics, Logs, Alarms, Dashboards
X-Ray	Distributed tracing for microservices
CloudTrail	API activity & auditing
Kinesis Data Firehose	Centralized logs to S3 / OpenSearch
Alarms & SNS	Alerts, notifications, incident response

DISASTER RECOVERY PLAN

COMPONENT	DR STRATEGY
S3	Cross-Region Replication
DynamoDB	Global Tables
Route 53	Health Checks + Failover
CloudFront	Multi-Region Origins
Media Services	Multi-AZ + Multi-Region
RPO / RTO	< 15 min / < 1 hr

COST OPTIMIZATION

- Use CloudFront caching aggressively, prepackage, deliver.
- Use recordings in S3 IA / Glaciers for scalability.
- Use optimize for low latency, availability, and quality.
- Secure content and protect user experience.
- Monitor everything and automate operations.

PRO TIP
Start with the user experience (latency, quality), then design the architecture to support it at scale.

EXAMPLE METRICS TO TRACK

- Concurrent viewers
- Chat message latency
- Buffer ratio / rebuffer events
- Transcoding queue depth
- Startup time (playback start)
- Origin fetch rate
- Error rate (%)
- Bandwidth usage (GB)

INTERVIEW QUESTIONS (EXAMPLE)

- 1 How would you achieve < 10s latency for live streaming?
- 2 How do you handle millions of concurrent viewers?
- 3 How would you implement adaptive bitrate streaming?
- 4 How do you secure your content from piracy?
- 5 How would you support real-time chat at scale?
- 6 How do you ensure high availability and disaster recovery?

KEY TAKEAWAYS

- Break streaming into ingest, process, package, deliver.
- Use managed AWS media services for scalability.
- Optimize for low latency, availability, and quality.
- Secure content and protect user experience.
- Monitor everything and automate operations.

CASE STUDY 6: MIGRATING ON-PREMISES TO AWS

PAGE
16

Design a secure, reliable, and cost-effective migration strategy for enterprise applications and data to AWS.

MIGRATION JOURNEY OVERVIEW



KEY REQUIREMENTS

- Minimize downtime and business disruption
- Secure data in transit and at rest
- Application compatibility & performance
- Data integrity and consistency
- Network connectivity and hybrid access
- Cost optimization and right-sizing
- Compliance and governance
- Rollback and disaster recovery strategy
- Operational readiness and monitoring

MIGRATION ASSESSMENT CHECKLIST

- Inventory of servers, applications, databases, and dependencies
- Performance and resource utilization
- Data volume, growth, and access patterns
- Licensing and compliance requirements
- Security, identity, and access controls
- Network topology and bandwidth needs
- Business criticality and downtime tolerance
- Application compatibility with AWS
- Cost estimation and TCO analysis

MIGRATION STRATEGIES (6Rs)

STRATEGY	DESCRIPTION	BEST USED WHEN
Rehost (Lift & Shift)	Move applications as-is to AWS with minimal changes.	Quick migration, minimal changes required.
Replatform (Lift, Tinker & Shift)	Make some optimizations without changing core architecture.	Improve scalability, performance.
Refactor (Re-architect)	Rebuild application using cloud-native services.	Maximize for cloud benefits and agility.
Repurchase	Replace with SaaS solution.	When a SaaS meets most business needs.
Retain	Keep on-premises.	App not ready / cost not justified.
Retire	Decommission unused or redundant applications.	Reduce cost and complexity.

HIGH-LEVEL MIGRATION ARCHITECTURE



AWS MIGRATION SERVICES



MIGRATION FLOW OVERVIEW

- Discover & assess on-prem infrastructure and applications.
- Classify applications (6Rs) and prioritize.
- Set up connectivity (Direct Connect / VPN).
- Prepare AWS landing zone (Accounts, VPCs, IAM, Guardrails).
- Migrate data (DMS, Snowball, DataSync).
- Migrate applications using best-fit strategy.
- Validate, test, and optimize in the cloud.
- Cutover using phased or big-bang approach.
- Optimize cost, performance, and security.

TECHNOLOGIES USED

- Compute**: EC2, ECS, EKS, Lambda
- Storage**: S3, EFS, EBS, FSx
- Database**: RDS, Aurora, DMS
- Migration**: AWS MGN, DMS, DataSync, Snowball, SCT
- Networking**: Direct Connect, VPN, Transit Gateway
- Security**: IAM, KMS, Shield, WAF
- Monitoring**: CloudWatch, CloudTrail, AWS Config

PHASED MIGRATION APPROACH

- Phase 1: Foundation**
Set up AWS landing zone, networking, security, and governance.
- Phase 2: Non-Critical Apps**
Migrate low-risk applications first.
- Phase 3: Core Applications**
Migrate business-critical workloads with minimal disruption.
- Phase 4: Optimization**
Refactor for cloud-native and optimize cost & performance.

DATA MIGRATION OPTIONS

- AWS DMS**
(Online migration with minimal downtime)
- AWS Snowball / Snowball Edge**
(For large data transfers)
- AWS DataSync**
(Fast, online data transfer)
- Amazon S3 Transfer Acceleration**
(For long-distance transfers)

NETWORKING CONSIDERATIONS

- AWS Direct Connect**
(Dedicated, consistent bandwidth)
- Site-to-Site VPN**
(Encrypted over internet)
- Transit Gateway**
(Centralized network management)
- VPC Peering / PrivateLink**
(Secure service connectivity)
- Route 53**
(DNS routing & failover)

APPLICATION MIGRATION PATTERNS

- Web Applications**
ALB + Auto Scaling + RDS
- Microservices**
ECS / EKS + ALB + (Aurora / DynamoDB)
- Databases**
DMS + RDS / Aurora
- File Services**
AWS FSx / EFS + DataSync
- Legacy Applications**
Rehost or Replatform on EC2

CUTOVER STRATEGIES

- Big Bang Cutover**
 - Faster migration
 - Higher risk
 - For non-critical systems
- Phased Cutover**
 - Lower risk
 - More control
 - Recommended for critical systems

VALIDATION CHECKLIST

- Functional testing passed
- Performance benchmarks met
- Security and access verified
- Data integrity validated
- Backup and DR tested
- Monitoring & alerts configured
- Rollback plan tested
- Stakeholders sign-off

RISK & MITIGATION

RISK	MITIGATION
Downtime	Use MGN/DMS, phased cutover, blue/green deployment.
Data Loss	Use replication, snapshots, and validation.
Security Breach	Encrypt data, least privilege access, security groups.
Cost Overrun	Right-sizing, budgets, usage monitoring.
Performance Issues	Load testing, auto scaling, cloud-native services.

COST OPTIMIZATION POST-MIGRATION

- Right-size instances & storage
- Use Savings Plans & Reserved Instances
- Leverage Spot Instances for non-critical jobs
- Enable S3 Lifecycle policies
- Use CloudFront & caching
- Monitor with AWS Cost Explorer & Budgets
- Tag resources and allocate costs

COMMON PITFALLS

- No clear migration strategy
- Ignoring network bandwidth needs
- Underestimating data transfer cost
- Not testing before production cutover
- Over-provisioning resources
- Ignoring security & compliance
- No rollback plan
- Lack of monitoring post-migration

REAL-WORLD EXAMPLE

A manufacturing company migrated 200+ VMs from on-premises to AWS using AWS MGN (Rehost) for quick lift & shift, then refactored critical apps to containers on EKS, migrated databases to Aurora using DMS, and reduced infrastructure cost by 35% while improving availability and performance.



INTERVIEW QUESTIONS (EXAMPLE)

- How would you assess and prioritize applications for migration?
- What are the pros and cons of rehosting vs refactoring?
- How do you ensure data consistency during migration?
- How would you minimize downtime during cutover?
- What AWS services would you use for database migration?
- How do you optimize cost after migration?

KEY TAKEAWAYS

- Assess first, migrate with the right strategy, optimize always.
- Use AWS native services to speed up migration.
- Security, networking, and monitoring are critical.
- Test, validate, and plan rollback before cutover.
- Think long-term: modernize and innovate after migration.

PRO TIP
Start small, learn fast, and iterate. Cloud migration is a journey, not a one-time project.

SENIOR SYSTEM DESIGN INTERVIEW QUESTIONS

PAGE
17

Real-world questions asked at top companies. Practice thinking out loud and designing scalable, reliable, and cost-effective systems on AWS.

HOW TO APPROACH ANY SYSTEM DESIGN QUESTION

- 1 Clarify Requirements**
Ask questions and define scope.
- 2 Define Assumptions**
State assumptions clearly.
- 3 Back-of-the-Envelope Estimation**
Do quick math; show your thinking.
- 4 High-Level Design**
Generate components and flow.
- 5 Deep Dive**
Discuss trade-offs and alternatives.
- 6 Non-Functional Requirements**
Scale, security, reliability, cost.
- 7 Wrap Up**
Summarize and suggest improvements.

NON-FUNCTIONAL REQUIREMENTS TO ALWAYS CONSIDER

- Scalability (Horizontal & Vertical)
- High Availability & Fault Tolerance
- Low Latency & High Performance
- Security & Compliance
- Data Consistency & Durability
- Cost Optimization
- Monitoring & Observability
- Maintainability & Operability
- Disaster Recovery

QUESTION CATEGORIES



SELECTED SYSTEM DESIGN QUESTIONS

- Design a URL shortener like bit.ly.
- Design a distributed caching system.
- Design a news feed system (e.g., Facebook News Feed).
- Design a chat application like WhatsApp.
- Design a real-time notification system.
- Design an e-commerce platform like Amazon.
- Design a payment processing system.
- Design a ride-sharing system like Uber.
- Design a food delivery system like DoorDash.
- Design a video streaming platform like Netflix.
- Design a live streaming platform (Twitch / YouTube Live).
- Design a recommendation system.
- Design a multi-tenant SaaS application.
- Design a data analytics pipeline.
- Design a global DNS service.

WHAT INTERVIEWERS LOOK FOR

- Structured thinking and clear communication
- Handling ambiguity and evolving requirements
- Trade-off analysis and decision making
- Depth in AWS services and best practices
- Scalability, reliability, and cost consciousness
- Security and operational excellence mindset

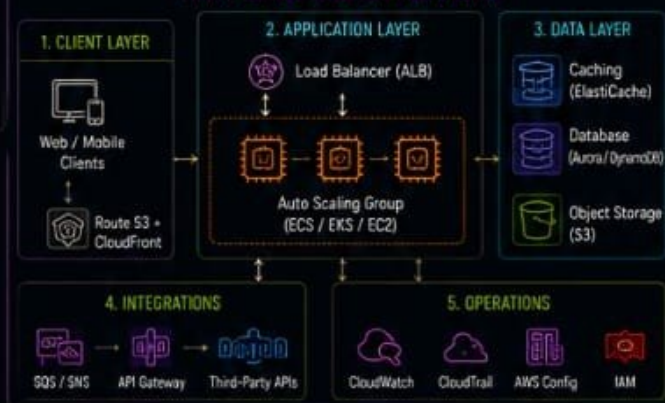
COMMON FOLLOW-UP PROMPTS

- How would you scale this system to 1 billion users?
- How would you handle failures?
- How would you reduce latency?
- How would you ensure data consistency?
- How would you make it cost-effective?
- How would you monitor and alert?
- How would you secure the system?
- What are the bottlenecks of your design?

ESTIMATION CHEAT SHEET

METRIC	RULE OF THUMB
QPS	Peak QPS = DAU × Avg actions / day + 86,400
Storage	Object size × Count × Replication factor
Traffic	Bandwidth = Data transferred / time
Latency	Cache: < 10ms DB: 10-20ms Cross-region: 70-150ms
Availability	Target 99.9%+ (3 nines = ~8.76 hrs/yr downtime)

HIGH-LEVEL DESIGN TEMPLATE



EXAMPLE ASSUMPTIONS TEMPLATE

- Users: 10M daily active users
- Peak QPS: 5,000
- Data per user: 1 KB
- Read:Write ratio: 90:10
- Availability: 99.99%
- Regions: us-east-1 (primary), us-west-2 (DR)

TRADE-OFF FRAMEWORK

DECISION	OPTION A	OPTION B	TRADE-OFF
Database	Aurora	DynamoDB	Strong consistency vs high scale & cost
Caching	ElastiCache	DynamoDB DAX	Performance vs complexity
Compute	ECS Fargate	EC2 Auto Scaling	Ease of use vs control & cost
Storage	S3 Standard	S3 Intelligent-Tiering	Cost vs access speed
Delivery	CloudFront	ALB	Edge caching vs dynamic routing

DESIGN PRINCIPLES TO APPLY

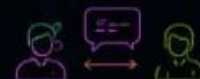
- Decouple components
- Single responsibility
- Stateless services
- Scale horizontally
- Design for failure
- Automate everything
- Measure and iterate

COMMUNICATION TIPS

- Think out loud and explain your reasoning.
- Draw neat diagrams and label everything.
- State assumptions clearly.
- Start simple, then iterate.
- Highlight trade-offs and justify decisions.
- Summarize at the end.
- Ask if the interviewer wants to go deeper.

SAMPLE CLOSING STATEMENT

To summarize, I designed a highly available, scalable, and secure system using AWS best practices. If I had more time, I would deep dive into the data model, security controls, and cost optimization strategies."



INTERVIEWER'S MINDSET

Interviewers are not looking for the "perfect" design. They want to see how you think, make trade-offs, and handle uncertainty like a senior engineer.



PRO TIP

Practice with a timer. Start with 45 minutes and try to complete a full design. Review real systems you use daily and think how you would design them on AWS.

50 ADVANCED AWS INTERVIEW QUESTIONS

FOR SENIOR ENGINEERS & SOLUTION ARCHITECTS

Practice these advanced questions to strengthen your system design, architecture, and operational excellence skills on AWS.

HOW TO USE THIS PAGE

- ✓ Pick a category and practice the questions.
- ✓ Think out loud. Draw diagrams.
- ✓ State assumptions and trade-offs.
- ✓ Ask clarifying questions.
- ✓ Discuss alternatives and optimizations.

PAGE
18

A. SYSTEM DESIGN (1-10)



- 1 Design a global URL shortening service like bit.ly.
- 2 Design a ride-sharing system like Uber.
- 3 Design a food delivery platform like DoorDash.
- 4 Design a video streaming platform like Netflix.
- 5 Design a chat application like WhatsApp.
- 6 Design a social media feed system (e.g., Facebook News Feed).
- 7 Design a real-time notification system.
- 8 Design a payment processing system.
- 9 Design a live streaming platform (Twitch / YouTube Live).
- 10 Design an e-commerce platform like Amazon.

B. SCALABILITY & PERFORMANCE (11-20)



- 11 How would you design for 10x traffic growth?
- 12 How would you scale a write-heavy database?
- 13 How would you handle a sudden traffic spike?
- 14 How would you design for low latency (< 50ms)?
- 15 How would you ensure high availability (99.99%+)?
- 16 How would you improve the performance of an API?
- 17 How would you optimize a slow-running SQL query?
- 18 How would you scale a system to 1 billion users?
- 19 How would you handle hot partitions in DynamoDB?
- 20 How would you design for massive read scalability?

C. RELIABILITY & RESILIENCE (21-30)



- 21 How would you design for fault tolerance?
- 22 How would you achieve zero RPO and near-zero RTO?
- 23 How would you design a cross-region disaster recovery solution?
- 24 How would you handle a complete AZ failure?
- 25 How would you ensure data durability for mission-critical data?
- 26 How would you prevent cascading failures?
- 27 How would you design idempotent APIs?
- 28 How would you handle retries and backoff strategies?
- 29 How would you build a highly available load balancer setup?
- 30 How would you design a system with graceful degradation?

D. DATA & STORAGE (31-35)



- 31 When would you choose Aurora over DynamoDB?
- 32 How would you model a relational database for a multi-tenant SaaS application?
- 33 How would you design a data lake on AWS?
- 34 How would you ensure data consistency across services?
- 35 How would you design a system with real-time analytics on streaming data?

E. SECURITY & COMPLIANCE (36-40)



- 36 How would you secure data in transit and at rest?
- 37 How would you implement least privilege access?
- 38 How would you design a multi-account security strategy?
- 39 How would you handle secrets and key management?
- 40 How would you ensure PCI-DSS compliance on AWS?

F. OPERATIONS & MONITORING (41-45)



- 41 How would you monitor a distributed system?
- 42 How would you set up centralized logging?
- 43 How would you design alerting to reduce noise?
- 44 How would you perform a blameless postmortem?
- 45 How would you automate infrastructure operations?

G. COST & OPTIMIZATION (46-48)



- 46 How would you reduce AWS costs without affecting performance?
- 47 How would you identify and eliminate waste in AWS?
- 48 How would you design a cost-aware architecture?

H. HYBRID & NETWORKING (49-50)



- 49 How would you connect an on-premises data center to AWS securely?
- 50 How would you design a hybrid cloud architecture that is highly available and secure?

INTERVIEWER EXPECTATIONS

- ☆ Clear communication and structured thinking
- ☆ Deep AWS services knowledge and trade-offs
- ☆ Attention to scalability, reliability, security, and cost
- ☆ Operational excellence and real-world experience
- ☆ Ability to justify decisions and handle ambiguity



EXAMPLE QUESTION BREAKDOWN

? Example: Design a video streaming platform like Netflix.

1. CLARIFY REQUIREMENTS



- Functional requirements
- Non-functional requirements
- Scale & growth
- Constraints & assumptions

2. HIGH-LEVEL DESIGN



- Major components
- Data flow
- Service boundaries
- Technology choices

3. DEEP DIVE



- Component design
- Data storage
- Scalability
- Reliability & fault tolerance

4. TRADE-OFFS



- Alternative approaches
- Pros & cons
- Cost vs performance
- Complexity vs reliability

5. OPTIMIZATIONS



- Performance tuning
- Cost optimization
- Security enhancements
- Operational excellence



TIPS TO SUCCEED

- ✓ Always start with requirements.
- ✓ Draw diagrams and communicate.
- ✓ Think about scale, failure, and cost.
- ✓ State assumptions clearly.
- ✓ Iterate and improve your design.



COMMON MISTAKES TO AVOID

- ✗ Jumping into design without clarifying requirements.
- ✗ Ignoring trade-offs and alternatives.
- ✗ Over-engineering too early.
- ✗ Not considering failure scenarios.
- ✗ Forgetting cost and operational complexity.



KEY TAKEAWAYS

- ✓ Senior interviews test your reasoning, not memorization.
- ✓ Design for scale, reliability, security, and cost.
- ✓ Always communicate and justify your decisions.
- ✓ Practice regularly and learn from real-world systems.
- ✓ Be an architect. Think beyond services.

PRODUCTION POSTMORTEMS

Learn from failures. Build resilient systems.

"Blameless postmortems drive learning. Learning drives reliability."

WHAT IS A POSTMORTEM?

A postmortem is a structured analysis of an incident to understand what happened, why it happened, and how to prevent it in the future. Focus on **SYSTEMS**, not people. 🧐

- ✓ Blameless
- ✓ Data-driven
- ✓ Action-oriented
- ✓ Future-focused

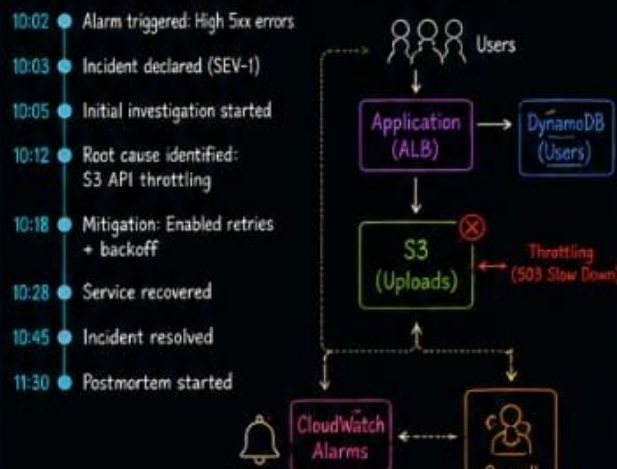
PAGE
19



POSTMORTEM PROCESS

- 1 **Detect & Declare**
Identify issue and declare incident. ⚠️
- 2 **Respond & Mitigate**
Restore service and reduce impact. ⚙️
- 3 **Investigate**
Collect data, timelines, logs, metrics. 🔍
- 4 **Root Cause Analysis**
Find the real underlying cause. 🎯
- 5 **Action Items**
Define improvements and owners. 📝
- 6 **Follow Up**
Track actions and verify improvements. 📊

INCIDENT TIMELINE EXAMPLE (S3 OUTAGE)



ROOT CAUSE ANALYSIS EXAMPLE

Problem: Users couldn't upload files.

5 WHYS Analysis

- 1 **Why?** S3 returned 503 Slow Down. Because too many PUT requests.
- 2 **Why?** No request throttling or retries. Because client didn't implement.
- 3 **Why?** High traffic spike during launch. Because we underestimated load.
- 4 **Why?** No load testing at that scale. Because test env was too small.
- 5 **Why?** No capacity planning process. Because we didn't have one.

Root Cause:
Lack of capacity planning and resilient client design. 🎯

REAL-WORLD OUTAGE CASE STUDIES

1. DYNAMODB HOT PARTITION

What happened? Traffic was unevenly distributed causing a hot partition. 🗄️

Impact: High latency and throttled requests.

Root Cause: Poor partition key design.

How we fixed it:

- ✓ Re-designed partition key
- ✓ Implemented auto-scaling
- ✓ Added DAX for caching

2. RDS FAILOVER FAILURE

What happened? Multi-AZ failover didn't happen due to storage issue. 🗄️

Impact: Database was unavailable for 18 min.

Root Cause: Storage full on standby instance.

How we fixed it:

- ✓ Enabled storage autoscaling
- ✓ Added CloudWatch alarms
- ✓ Regular failover testing

3. CLOUDFRONT MISCONFIGURATION

What happened? Incorrect cache behavior caused stale content. ☁️

Impact: Users saw outdated data.

Root Cause: Cache TTL too high.

How we fixed it:

- ✓ Adjusted TTL settings
- ✓ Added cache invalidation
- ✓ Implemented versioned assets

4. IAM PERMISSION ISSUE

What happened? Application lost access to S3 after IAM policy change. 🔒

Impact: File uploads failed.

Root Cause: Overly restrictive IAM policy.

How we fixed it:

- ✓ Reviewed least privilege
- ✓ Added IAM Access Analyzer
- ✓ Implemented CI/CD policy checks

POSTMORTEM TEMPLATE

- Summary
- Impact
- Timeline
- Root Cause
- What Went Well
- What Didn't Go Well
- Action Items
- Preventive Measures
- Lessons Learned
- Appendices (Graphs, Logs) ...

WHAT WENT WELL vs WHAT DIDN'T GO WELL

- | What Went Well ✓ | What Didn't Go Well ✗ |
|----------------------------|------------------------|
| ✓ Quick incident detection | ✗ No load testing |
| ✓ Good communication | ✗ Missing alarms |
| ✓ Fast mitigation | ✗ Poor error handling |
| ✓ Runbooks helped | ✗ No capacity planning |



COMMON ACTION ITEMS

- 📅 Improve monitoring & alerting
- 📅 Add retries with exponential backoff
- 🔌 Implement circuit breakers
- ⚙️ Enhance auto-scaling policies
- 📅 Improve capacity planning
- 🧪 Run chaos engineering tests
- 📅 Update runbooks & documentation
- 🧑 Conduct regular game days

POSTMORTEM PRINCIPLES

- 👥 **Blameless Culture**
Focus on systems, not people.
- 👁️ **Transparency**
Share openly and honestly.
- 🧠 **Learning Mindset**
Every incident is a lesson.
- 🎯 **Action Driven**
Ideas without action = noise.
- 📊 **Follow Through**
Track actions to completion.

TOOLS THAT HELP

- 📊 CloudWatch (Metrics & Alarms)
- 🔍 X-Ray (Tracing)
- 📄 CloudTrail (Audit Logs)
- ⚙️ AWS Config (Change Tracking)
- 👤 OpsCenter (Incident Mgmt)
- 📞 PagerDuty (On-call Alerts)

BLAMELESS POSTMORTEM MINDSET



KEY TAKEAWAYS

- ✓ Failures are inevitable, learning is optional.
- ✓ Great engineers learn faster from failures.
- ✓ Postmortems turn incidents into improvements.
- ✓ Build systems that can fail and recover.
- ✓ Design for observability, resilience, and scale. 🏆



PRO TIP

After every incident, ask: "What will we do differently next time?" Small improvements after every incident lead to huge reliability.

“The goal is not to avoid failures, but to get better at recovering.”



FINAL TAKEAWAYS & INTERVIEW GAME PLAN

Think like an Architect. Solve like an Engineer. Deliver like a Leader. ★

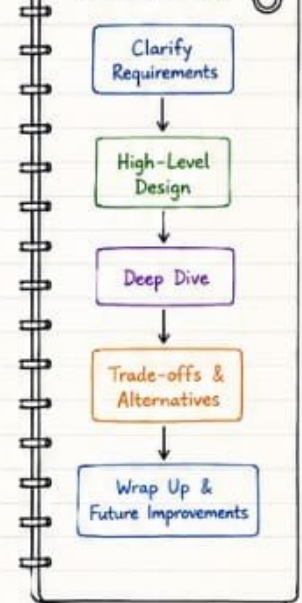
1. PRINCIPLES THAT GUIDE EVERY GREAT DESIGN

-  **Customer Obsession** Start with the user and work backwards.
-  **Simplicity** Keep it simple. Avoid unnecessary complexity.
-  **Operational Excellence** Automate, monitor and improve continuously.
-  **Security** Secure by design. Least privilege. Encrypt everything.
-  **Reliability** Build fault-tolerant systems that recover automatically.
-  **Performance Efficiency** Use the right services. Scale appropriately.
-  **Cost Optimization** Optimize for long-term value, not just short-term cost.
-  **Sustainability** Build efficient systems that reduce waste.

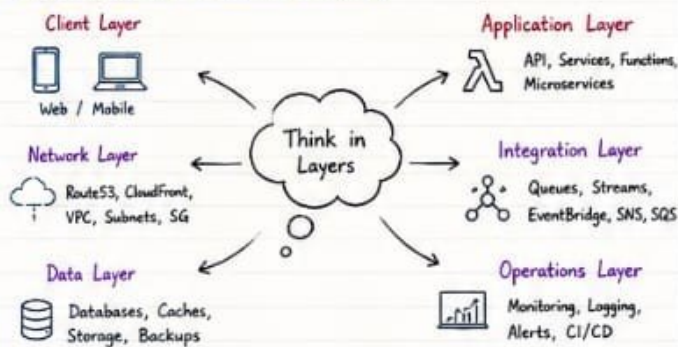
2. THE SYSTEM DESIGN INTERVIEW GAME PLAN

- ① **Clarify Requirements**
 - Ask the right questions.
 - Define scope, constraints, and assumptions.
- ② **Define & Prioritize**
 - Identify must-have vs nice-to-have.
 - Set priorities with the interviewer.
- ③ **High-Level Design**
 - Draw the big picture.
 - Explain the flow and major components.
- ④ **Deep Dive**
 - Zoom into critical components.
 - Discuss data, scale, security, and trade-offs.
- ⑤ **Trade-offs & Alternatives**
 - Compare approaches honestly.
 - Explain why you chose your design.
- ⑥ **Wrap Up**
 - Summarize your design.
 - Discuss future improvements.

INTERVIEW FLOW



3. ARCHITECT'S MENTAL MODEL



4. COMMON DESIGN TRADE-OFFS

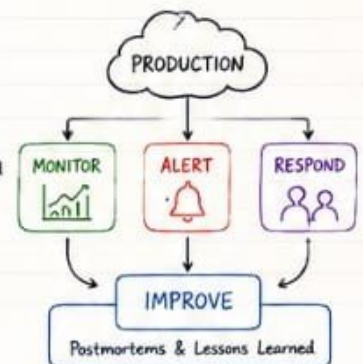
Trade-Off	Option A	Option B	Consider When
Consistency vs Availability	RDS Multi-AZ (Consistency)	DynamoDB (Availability)	Choose based on use case criticality
Cost vs Performance	S3 Standard	S3 Infrequent Access / Glacier	Balance speed and storage cost
Simplicity vs Scalability	Monolith	Microservices	Start simple, scale when needed
Managed vs Control	AWS Managed Services	Self-Managed (Open Source)	Speed vs customization and cost
Short Term vs Long Term	Quick & Dirty Solution	Well-Architected Solution	Think long-term value and maintenance

5. AWS SERVICES QUICK RECALL MAP



6. PRODUCTION READINESS CHECKLIST

- ☒ High availability across AZs
- ☒ Auto scaling configured
- ☒ Backups and snapshots enabled
- ☒ Monitoring and alerting in place
- ☒ Logging and tracing enabled
- ☒ Security groups & NACLs configured
- ☒ IAM least privilege access enforced
- ☒ Secrets encrypted and rotated
- ☒ Cost alarms and budgets set
- ☒ Disaster recovery plan tested
- ☒ Runbooks and SOPs documented



7. BEFORE YOU ENTER THE INTERVIEW ROOM

- ☆ Be confident, not over-confident.
- ☆ Listen carefully.
- ☆ Communicate your thinking clearly.
- ☆ Draw clean diagrams.
- ☆ Make reasonable assumptions.
- ☆ Start simple.
- ☆ Iterate your design.
- ☆ Handle feedback gracefully.
- ☆ Show trade-offs.
- ☆ End with improvements.



You Got This! ♥

8. FINAL WORDS

"Every outage teaches a lesson.
Every design shapes the future.
Every day you get better,
you become the architect others
rely on."

